

LDCI2027: Light-Duty CI-ICE 2027

PROJECT ASSESSMENT 3

DEVELOPMENT OF AI-BASED OPERATION STRATEGY

REPORT 1

OCTOBER 8, 2025

FVV project 1507

Abstract

This work presents the development and implementation of a predictive controller based on simulations generated by two complementary recurrent neural networks, named ICE (internal combustion engine) and PG (hybrid powertrain). These neural networks replace complex physical simulations, allowing for fast and accurate inferences of the hybrid vehicle system’s dynamic behavior. Initially, an incremental neural network-based controller was developed, focused on maintaining a constant vehicle speed, validating multiple strategies to stabilize training, handle physical constraints in a differentiable way, and improve numerical robustness. Although satisfactory results were obtained in certain operating scenarios, the limitations found motivated the consideration of a future implementation using deep reinforcement learning (RL). The methodological techniques validated in this study, such as curriculum learning, soft functions for handling physical constraints, and dynamic training stabilization, offer a solid and transferable foundation for developing a more robust and higher-performing controller in future stages.

Index

Abstract	i
List of Figures	iv
Symbols	v
1 Introduction	1
2 Project Version History	2
3 Simulator Used	4
3.1 Important Model Characteristics	4
3.2 Model Inputs and Outputs	4
4 Adaptation of Models for Efficient Inference	5
4.1 Integration of the model into the simulator	5
4.2 Validation of the implementation	5
5 Controller	9
5.1 First problem: combustion-only vehicle (cruise control)	10
5.2 Second problem: combustion-only vehicle (cruise control) with V2 models	21
6 Objective and Paradigm Shift (from “Gradients-based” to RL)	27
6.1 Recurrence Architectures and Model Choice	27
7 RL Model: Implications and Conceptual Design	28
8 Selected Algorithm: Recurrent TD3 (justification and configuration)	30
9 Architectures: Actor RNN and Critics RNN	31
9.1 Actor RNN	31
9.2 Critics RNN	32
9.3 Focused Comparison: from Initial to Current Architecture (actor)	32
10 Optimizing the Training Process: Efficiency and Stability	33
10.1 Memory Management: The Overlapping Windows Buffer	33
10.2 Accelerating Experience: Actor-Learner Architecture	34
11 Network Update: Actor and Critic Loss Functions	34
11.1 Critic Loss	34
11.2 Actor Loss	35
12 Summary of System Architecture and Modularity	35
13 Discussion of Results and Limitations	36
A transition_function_model class	38
B Experimentation Script with test	40
C Experimentation Script with Case Studies	44
D Controller Architecture V1	47
E Controller Training Script V1	47
F Controller Architecture V2	50

G Custom Scalers Collage	52
H Controller Training Script V2	54

List of Figures

1	Neural Network Model M1.1	4
2	Case 1, comparison of NO_x emissions profile: (a) Original vs. (b) Custom implementation.	7
3	Case 2, comparison of ICE torque profile: (a) Original vs. (b) Custom implementation.	8
4	Case 3, comparison of SOC and velocity profiles: (a) Original vs. (b) Custom implementation.	9
5	Information flow in the ICE-PG model-based controller	10
6	Controller behaviour with V1 models at a target speed of 70 km/h.	18
7	Controller behaviour with V1 models at a target speed of 10 km/h.	19
8	Controller behaviour with V1 models at a target speed of 130 km/h.	20
9	Controller behaviour with V2 models at a target speed of 70 km/h.	25
10	Controller behaviour with V2 models at a target speed of 20 km/h.	26

Symbols

Description	Python Variable	Units	Range
Engine speed	ice_sp	rpm	0 – 4500
Fuel mass	mf	mg	3 – 70
Ambient temperature	T_amb.K	K	250 – 310
Ambient pressure	p_amb.bar	bar	0.8 – 1.1
Influence on turbine efficiency	vtg	%	23 – 90
ICE torque	torque	Nm	–50 – 300
Nitrogen oxides	Nox	mg/s	-
Nitrogen dioxide	NO2_out.m_gps	mg/s	-
Carbon monoxide	CO	mg/s	-
Carbon dioxide	CO2_out.m_gps	mg/s	-
EM2 torque	EM2	Nm	–421 – 421
Brake percentage	brk	%	0 – 100
State of Charge	soc	%	0 – 100
Car velocity	vel_out	km/h	-

1 Introduction

The primary objective of this report is to show the developments made in the LDCI project task regarding an advanced predictive controller for vehicles with hybrid propulsion systems. The propulsion system behaviour is based on simulations provided by two complementary recurrent neural networks, identified as ICE (internal combustion engine model) and PG (hybrid powertrain model). These neural networks offer a fast, precise, and efficient representation of the vehicle system's dynamic behavior, replacing traditional physical simulations and allowing a faster evaluation of advanced control strategies.

The task strategy was divided into two main phases. In the first stage, a controller based on supervised learning was designed and implemented, using an incremental LSTM recurrent neural network. During this phase, specific training challenges were addressed, such as numerical stabilization, differentiable handling of physical constraints, and dynamic adaptation of the learning rate. Although positive results in terms of stability and precision were obtained in certain operating ranges, important limitations were also identified, which indicated the need to consider more robust approaches.

In a second phase, the controller's architecture was adapted and improved to meet the new constraints and characteristics of the updated neural networks (ICE V2.1 and PG V2.3). This stage allowed for the definitive validation of the proposed methodologies, demonstrating a significant improvement in the controller's stability and physical coherence, although limitations still persist at high speeds and under extreme operating conditions.

Finally, the results obtained provide valuable knowledge for later stages of the project. The validation of techniques such as curriculum learning, soft activations for physical constraints, training stabilization through gradient clipping, and the dynamic reduction of the learning rate are key elements that will allow for the future development of a controller based on deep reinforcement learning, capable of simultaneously optimizing multiple control objectives, such as the precision of speed tracking and the reduction of polluting emissions, ensuring robust performance across the entire operating range of the hybrid vehicle.

2 Project Version History

During the execution of this task, UPC has received both modifications to key concepts and new versions of the models (**PG** and **ICE**). This has led to frequent revisions and adjustments in several task sections, re-evaluations of technical solutions, and the need to re-run previously completed simulation and validation processes.

Below are the different model versions used, along with a brief explanation of how each one has influenced the task's development.

Version 1 (V1)

ICE Model M1

- **Inputs:** `ice_sp`, `mf`, `T_amb_K`, `p_amb_bar`, `vtg`
- **Outputs:** `torque`, `N0x`, `CO`

PG Model M1.1 without EM1 torque

- **Inputs:** `ice_sp`, `EM2`, `torque`
- **Outputs:** `vel_out`, `SoC`

This is the original version with which this task started.

Version 2 (V2)

Conceptual Change in the ICE Model Implementation

Due to the valid range of the internal combustion engine (ICE) neural network, which is only valid between 900 and 4000 rpm, a restriction is implemented in the execution:

When the output of `ice_sp` is less than 900 rpm, the conditions `mf=0` and `torque=0` are manually imposed, thus preventing the neural network from working outside its operational range.

Affected Sections:

- Section: *Controllers*
- Sub-section: *Important Model Characteristics*

Version 2.1 (V2.1)

New version of the ICE model (ICE_Model_Update_01)

An improved version of the ICE model is incorporated, in which both the inputs and outputs are modified, simplifying its structure and improving its performance.

- **Modified Inputs:** `ice_sp`, `mf`, `T_amb_K`, `p_amb_bar` (`vtg` is removed)
- **Expanded Outputs:** `torque`, `N0x`, `N02_out_m_gps`, `CO`, `CO2_out_m_gps`

Affected Sections:

- Sections: *Adaptation of models for efficient inference*, *Controllers*
- Sub-section: *Model Inputs and Outputs*

Version 2.2 (V2.2, final version)

New version of the PG model (PG_v2)

The definitive implementation of the PG model, which offers better predictive and operational performance compared to the initial version.

Affected Sections:

- Section: *Controllers*
- Sub-section: *Important Model Characteristics*

Note on Document Organization

Given that all these updates have occurred throughout the task's development, each change has involved a considerable workload, including code modifications, methodological adjustments, and complete re-executions of previous processes.

However, in this report, only the results and decisions of the initial (V1) and final (V2.2) versions are described in detail, as they represent the most significant phases of the task's development. The intermediate versions (V2 and V2.1) are mentioned only to provide context on the task's evolution.

3 Simulator Used

For this task, a previously developed simulator was used at the beginning, capable of representing the dynamic behavior of the internal combustion engine in a realistic manner and at a low computational cost.

The **PG+ICE** model was created during the *New Model M1* phase as part of an effort to replace complex physical simulations with *surrogate* models based on neural networks.

3.1 Important Model Characteristics

- **Trajectory Representation:** the PG+ICE model employs stateful LSTM neural networks to predict time series. This structure allows it to capture the dynamic behavior of the system realistically.
- **Systems:** it consists of two independent models, one for the internal combustion engine (ICE) and one for the powertrain (PG). Both models were trained with millions of data points generated by previously validated physical simulations.
- **Data Generation in Transient Cycles:** the training data were generated by simulating trajectories of 1800 seconds, with a time step of 0.5 s (2 Hz).
- **Computational Efficiency:** the neural networks replace complex numerical models, allowing for fast inferences. This drastically reduces simulation times and enables the training of agents.
- **Stability Validation:** the models were trained and validated within physically plausible operating ranges, which ensures stable simulations and coherent results.
- **ICE Lower Limit (V2):** if `ice_sp < 900`, `mf = 0` and `torque = 0` are enforced to prevent out-of-range inferences.

3.2 Model Inputs and Outputs

The PG+ICE simulator is composed of two main functions, which represent the sub-models of the internal combustion engine (ICE) and the powertrain (PG), as shown in Figure 1.

- **ICE Sub-model**
 - **Inputs:** `ice_sp`, `mf`, `T_amb_K`, `p_amb_bar`
 - **Outputs:** `torque`, `NOx`, `NO2_out_m_gps`, `CO`, `CO2_out_m_gps`
- **PG Sub-model**
 - **Inputs:** `ice_sp`, `EM2`, `torque`, `brk`
 - **Outputs:** `vel_out`, `SoC`

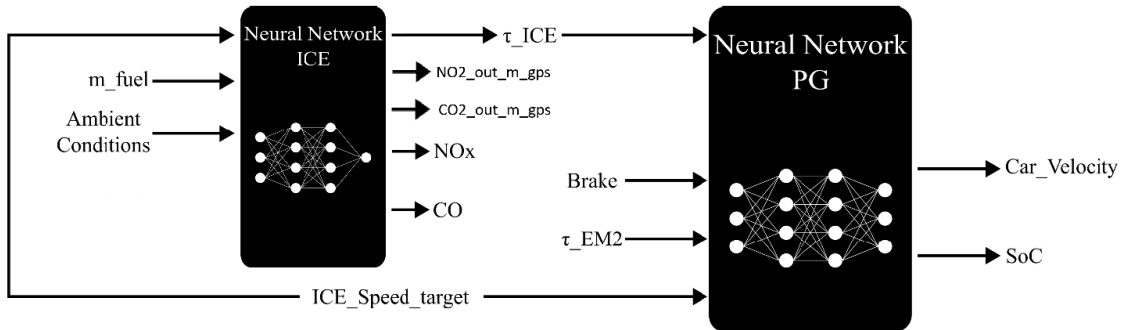


Figure 1: Neural Network Model M1.1

4 Adaptation of Models for Efficient Inference

4.1 Integration of the model into the simulator

To facilitate the use of the **PG** and **ICE** models, a class called `transition_function_model` was designed. This class encapsulates all the necessary elements to perform inferences in a simple and efficient way. This class allows the models to be invoked through direct functions, such as:

```
predict_ice(Speed_rpm, m_fuel_mg, T_amb_K, p_amb_bar)
```

To be functional within the simulator, the implementation had to comply with the following requirements:

- Automatically scale and unscale input and output variables using the scalers trained in previous project phases.
- Maintain the internal context of the LSTM models, which allows for the continuous simulation of temporal trajectories.
- Allow the internal state of the models to be reset, which is necessary when starting a new simulation.
- Allow for gradient propagation for use in differentiable training environments.

To do this, the class is initialized with the pre-trained models and their respective scalers. Internally, the models are cloned to avoid unwanted modifications, and the scaler parameters are converted to TensorFlow tensors for more efficient execution.

The main methods of the class are:

- `reset_models()`: resets the internal states of the LSTM models and returns the auxiliary variables to their initial state.
- `predict_ice(...)`: performs the inference of the ICE sub-model from physical input variables, returning the unscaled values of `torque`, `NOx` and `CO NO2_out_m_gps`, `CO2_out_m_gps`.
- `predict_PG(...)`: executes the PG sub-model, predicting the `vel_out` and the `SoC`.
- `predict(...)`: executes both sub-models in a coordinated manner, returning the full output of the hybrid system in a single call.

This integration encapsulates all the necessary logic into a single reusable structure, which simplifies the massive execution of simulations both in offline mode and during the training of reinforcement learning agents. In addition, the implementation is prepared to be differentiable, which allows its use in environments where gradient backpropagation is required.

The complete code for the `transition_function_model` class is shown in Appendix A.

4.2 Validation of the implementation

To ensure that the `transition_function_model` class developed in this project faithfully reproduces the expected behavior of the ICE and PG models, two complementary validation methods were designed:

- **Method 1 – Quantitative validation:** numerically compares the results obtained with this class against the original script (`Modular.py`) using real test datasets.
- **Method 2 – Qualitative reproduction of boundary scenarios:** visually analyzes whether the dynamic behavior of the simulated system is consistent with the reference cases defined by the original developers.

Both methods allow for the confirmation of the equivalence between versions and the detection of possible deviations introduced by the new implementation.

4.2.1 Method 1 – Quantitative validation with a test dataset

To verify that the adaptation of the simulator is correct, the `Modular.py` script, generated in the previous phase of the project, was reused. This script allows for the execution of test trajectories and the calculation of error metrics between the outputs generated by the models and the real outputs.

First, an execution was performed with the original (*vanilla*) models, i.e., those directly provided by the previous phase without any type of modification or custom integration. The results obtained were as follows:

```
versio b:
ICE_Torque:      MAE=1.4973 | MAPE=2.82%          | MaxAE=74.0598 | MaxRE=1462.54%
NO_out_m_pred:   MAE=0.0000 | MAPE=0.19%          | MaxAE=0.0017 | MaxRE=378.57%
NO2_out_m_pred:  MAE=0.0000 | MAPE=0.50%          | MaxAE=0.0000 | MaxRE=837.31%
CO_out_m_pred:   MAE=0.0000 | MAPE=0.17%          | MaxAE=0.0012 | MaxRE=69.09%
CO2_out_m_pred:  MAE=0.0000 | MAPE=0.57%          | MaxAE=0.0035 | MaxRE=920.52%
Car_Speed:       MAE=3.0239 | MAPE=14373.23%      | MaxAE=12.5154 | MaxRE=6133030.97%
```

Next, the script code was adapted to use the `transition_function_model` class developed in the current task, maintaining the same input trajectories and test configurations. The results were:

```
versio b:
ICE_Torque:      MAE=1.4973 | MAPE=2.82%          | MaxAE=74.0598 | MaxRE=1462.54%
NO_out_m_pred:   MAE=0.0000 | MAPE=0.19%          | MaxAE=0.0017 | MaxRE=378.53%
NO2_out_m_pred:  MAE=0.0000 | MAPE=0.49%          | MaxAE=0.0000 | MaxRE=837.31%
CO_out_m_pred:   MAE=0.0000 | MAPE=0.20%          | MaxAE=0.0012 | MaxRE=100.00%
CO2_out_m_pred:  MAE=0.0000 | MAPE=0.57%          | MaxAE=0.0035 | MaxRE=920.39%
Car_Speed:       MAE=3.0133 | MAPE=15531.23%      | MaxAE=12.4422 | MaxRE=3949470.00%
```

Given that the ICE and PG models are stochastic LSTM networks, the results obtained with the new implementation are considered equivalent to the originals. The small differences observed are within the expected margin and do not alter the overall validity of the model.

Therefore, it is concluded that the proposed adaptation correctly reproduces the behavior of the trained models and is suitable for use in simulations and agent training.

The complete code for this adaptation can be found in Appendix B.

4.2.2 Method 2 – Reproduction of boundary scenarios

Although in *Method 1* it was shown that the results obtained with the `transition_function_model` class matched those of the original script, very high absolute and relative errors were observed in some variables (especially speed and SoC). For this reason, a second, more visual approach was designed, focused on reproducing extreme dynamic trajectories, with the objective of verifying the model's robustness and qualitative consistency.

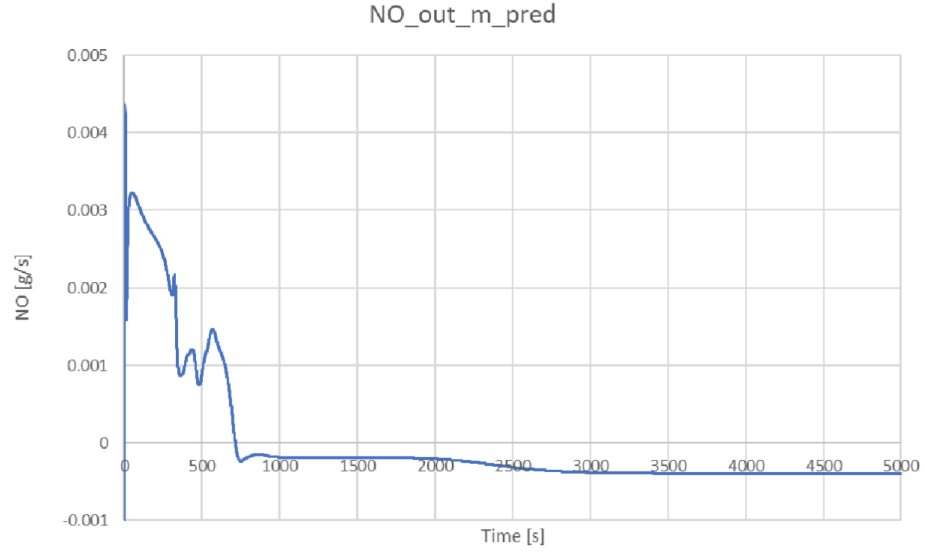
To do this, the experiments presented in the document *Model_Discussion_and_Limitations.pdf*, provided as a reference during development, were replicated. This document presents three specific scenarios used by the developers to illustrate and validate the behavior of the ICE and PG models. In this task, these same cases were reused as a qualitative validation test.

Below, the three reproduced scenarios are described, along with the input values used and the comparative graphs between the original results and those generated by the implementation developed in this task.

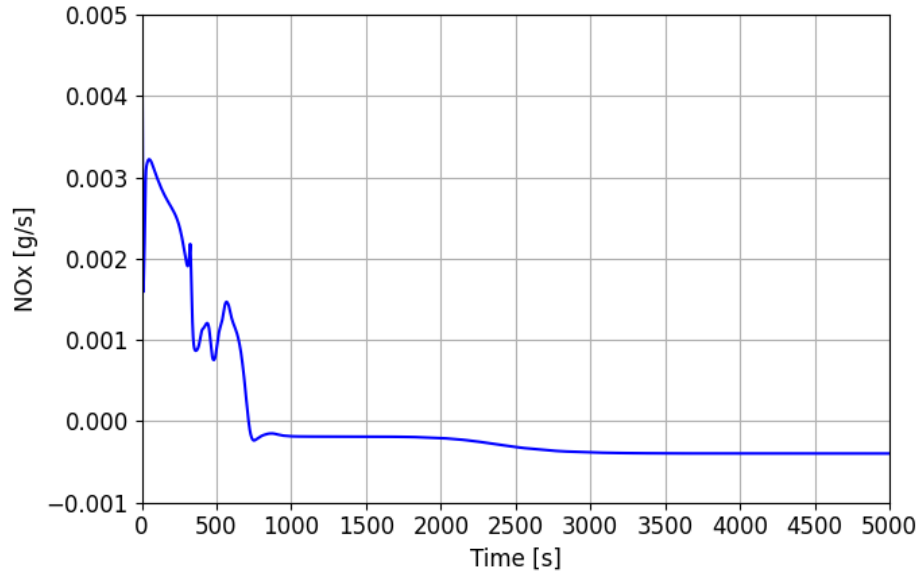
Case 1 – Low load and low speed

- ICE_Speed_rpm = 1000
- Fuelmass_mg = 3
- EM2_Torque_Nm = 0

- Brake_frac = 0
- SoC_Start_frac = 0.5



(a) Original

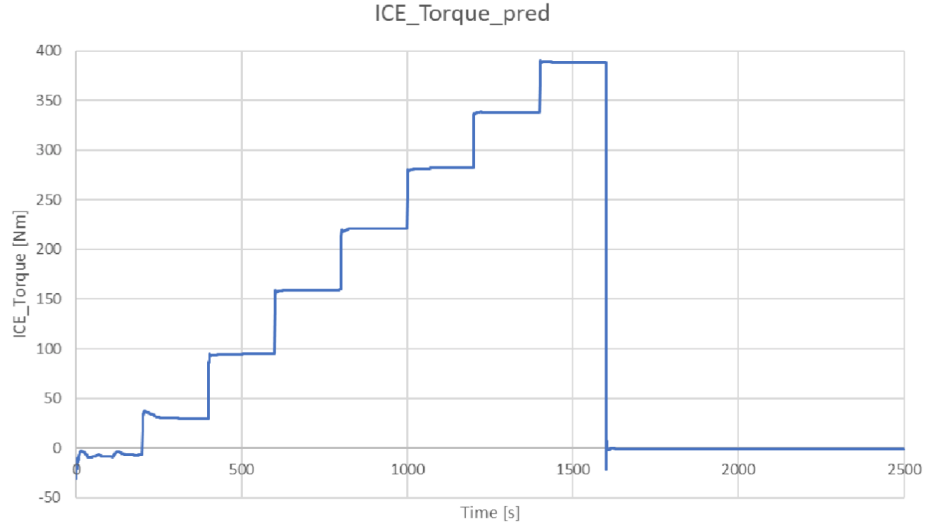


(b) Custom implementation

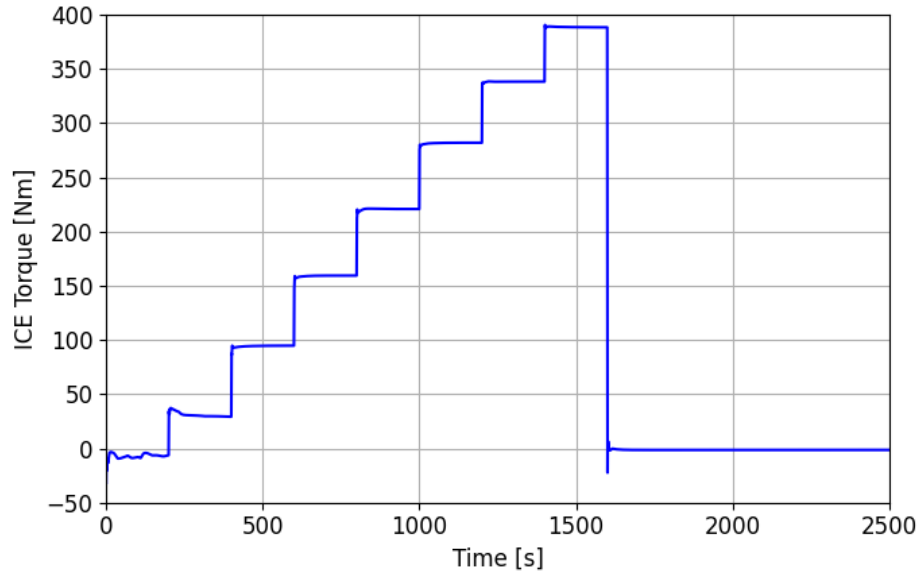
Figure 2: Case 1, comparison of NO_x emissions profile: (a) Original vs. (b) Custom implementation.

Case 2 – Load sweep at medium speed

- ICE_Speed_rpm = 2500
- Fuelmass_mg = [3, 10, 20, 30, 40, 50, 60, 70, 5]
- EM2_Torque_Nm = 0
- Brake_frac = 0
- SoC_Start_frac = 0.5



(a) Original



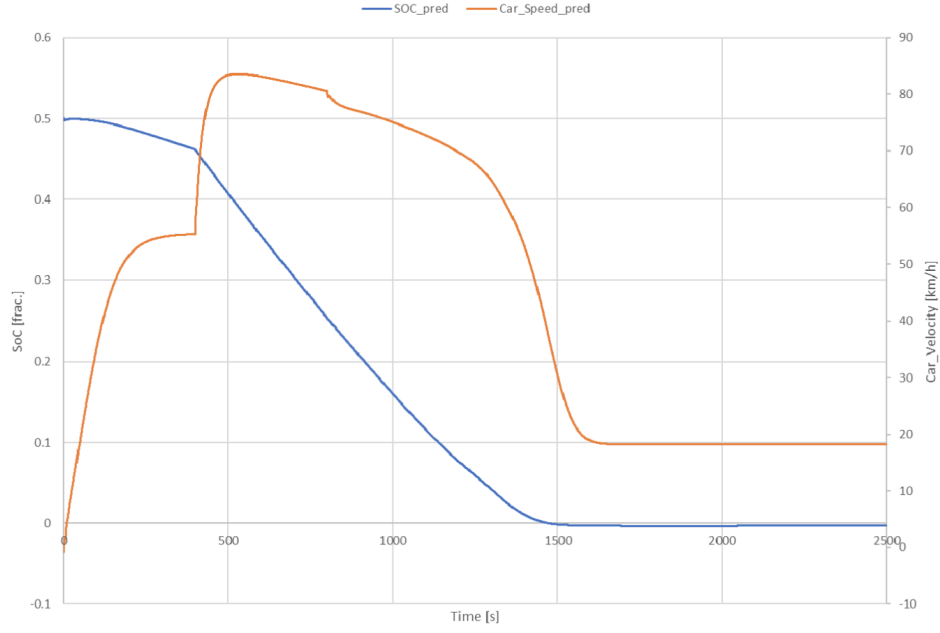
(b) Custom implementation

Figure 3: Case 2, comparison of ICE torque profile: (a) Original vs. (b) Custom implementation.

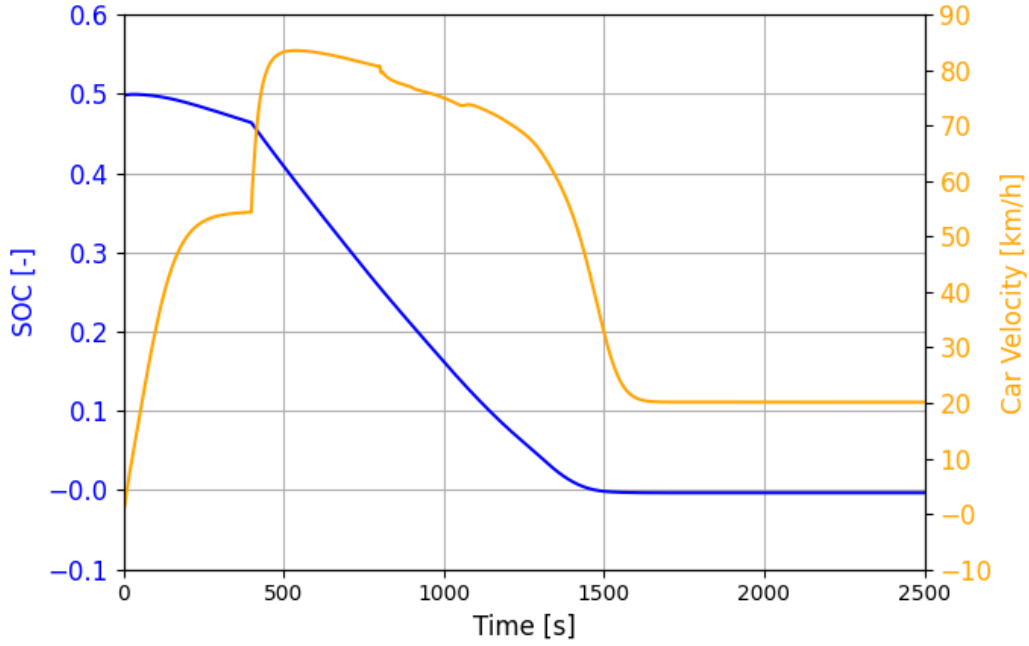
Case 3 – Purely electric traction (ICE off)

- ICE_Speed_rpm = 0
- Fuelmass_mg = 0
- EM2_Torque_Nm = [50, 250, 400]
- Brake_frac = 0
- SoC_Start_frac = 0.5

As can be seen in the Figures 2 to 4, the results obtained using the `transition_function_model` class faithfully reproduce the reference behavior described in the original document, which reinforces the validity of the implementation. The observed deviations are minimal and do not affect the overall analysis of the model.



(a) Original



(b) Custom implementation

Figure 4: Case 3, comparison of SOC and velocity profiles: (a) Original vs. (b) Custom implementation.

The code that generates these scenarios is included in Appendix D.

5 Controller

The central element of this work is a **predictive controller** designed to calculate, in real time, the input signals that feed the *digital twins* of the internal combustion engine (**ICE**) and the hybrid powertrain (**PG**). These digital twins provide a fast and faithful representation of the physical system, which allows for the evaluation of advanced control strategies in a safe and efficient manner.

The main objective of the controller is twofold: on one hand, to get the vehicle's speed to follow the desired setpoint as faithfully as possible; on the other, to minimize the polluting emissions generated during driving. This trade-off is formalized by a cost function that combines the speed error with the total predicted emissions (1):

$$\min_{u_0, \dots, u_{N_p-1}} J = \sum_{k=0}^{N_p-1} \left[w_v (v_{\text{ref}}(k) - v(k))^2 + \sum_{g \in \{\text{NO}_x, \text{NO}_2, \text{CO}, \text{CO}_2\}} w_g \dot{m}_g(k) \right] \quad (1)$$

where J is the cost function evaluated over a prediction horizon N_p . The first term quantifies the speed tracking error, with $v_{\text{ref}}(k)$ being the reference speed and $v(k)$ the predicted speed at instant k , weighted by the weight w_v . The second term corresponds to the weighted sum of the emissions of the different pollutants g , where $\dot{m}_g(k)$ represents the mass flow (in mg/s) of each pollutant (NO_x , NO_2 , CO and CO_2) and w_g is the specific weight associated with each of them.

The final ambition of the project is to implement a controller based on *deep reinforcement learning* that autonomously discovers the optimal policy. Nevertheless, in a first phase, the decision was made to fully leverage the capabilities of a **predictive model trained with neural networks**. **This preliminary approach allows for testing complex and innovative strategies and offers a solid baseline for evaluating whether it is worth incorporating a reinforcement learning agent, considering its greater complexity and implementation cost.** If the benefits justify the effort, the task will advance to the stage where the reinforcement agent uses the digital twins to explore a wider action space and potentially surpass the current performance.

Although classic predictive control and reinforcement learning strategies differ considerably in their methods, the general flow of the controller remains constant. First, the controller determines the new inputs that will be supplied to the digital models: ice_sp , m_f , $T_{\text{amb}K}$, $p_{\text{amb}bar}$ and $EM2$. Next, inference is performed on the ICE model using only the variables that correspond specifically to it, thus obtaining the engine torque (**torque**), which will be indispensable for the subsequent inference of the PG model, as well as the generated emissions: NO_x , NO_2 , CO and CO_2 . Finally, using the previously calculated engine torque along with the electrical input $EM2$, inference is performed on the PG model, obtaining the vehicle's speed (**vel.out**) and the battery's state of charge (**SoC**) (Figure 5).

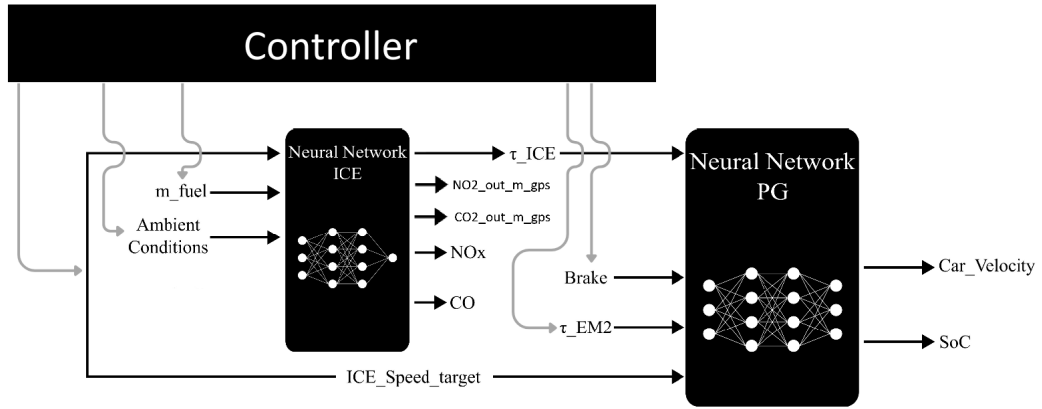


Figure 5: Information flow in the ICE-PG model-based controller

5.1 First problem: combustion-only vehicle (cruise control)

As a first step to address the control problem, a simplified scenario was defined, focused exclusively on the combustion engine, postponing the complexity of the hybrid system. The goal was to develop a cruise controller capable of maintaining a constant speed of 70 km/h, without considering, in this initial phase, the optimization of emissions.

For this analysis, **Version 1 (V1)** of the models, described in the section *Version History*, was used. To isolate the behavior of the combustion engine and simulate a non-hybrid vehicle, the following conditions were established:

- Simulations of **600 seconds** were performed.
- The electric motor torque, τ_{EM2} , was kept at **zero** throughout the simulation. Under this premise, and if the models are consistent, the state of charge of the battery (SoC) should not vary.
- The reference speed (**Car_Velocity_reference**) was fixed at a **constant value of 70 km/h**.

The general control problem consisted of designing a controller that, given a desired speed, would determine the actions $\{m_f(k), \text{vtg}(k), \text{brk}(k), \text{ice_sp}(k)\}$ that, when introduced into the models, would minimize a cost function.

5.1.1 Key Design Decisions

To proceed in a structured manner, a series of constitutive decisions were defined for the controller’s design. The choices detailed in this section are structural and indispensable components.

1. Definition of the loss function The first methodological decision was to define the loss function. Since the objective of this problem was exclusively speed control, the cost function naturally focused on the tracking error. This meant adopting only the left term of Equation (1).

Once the objective was established, the metric for quantifying this error was debated. Although options like the absolute error (MAE) were considered, the **mean squared error (MSE)** was chosen for its theoretical advantages.

The MSE loss function is continuously differentiable and, by quadratically weighting the errors, penalizes significant deviations more. This favors more stable convergence during training and helps avoid controller oscillations near the optimal value. The loss at each time step (**step_loss**) was defined as:

`step_loss = tf.square(vel_target - vel_out)`

2. Definition of the state space (controller inputs) Next, the input variables for the controller were decided. Initially, a very simple solution was chosen, using only the current speed and the desired speed. This initial approach had the advantage of a reduced state space, but it was quickly observed that this configuration suffered from high variance. For example, when the vehicle was traveling at 68 km/h and needed to reach 70 km/h, if the controller only received these two speeds, it did not have explicit information about the current level of injected fuel (m_{fuel}). This could lead to the solution being to increase the fuel mass even more, when in reality, it might have been more optimal to act on other variables like the VTG or the **ICE.Speed.target**. Therefore, the first clear conclusion was to expand the state space to the following variables: $\{\text{vel_target}, v(t), m_f(t), \text{vtg}(t), \text{brk}(t), \text{ice_sp}(t)\}$.

3. Definition of the output format (controller actions) The decision on how the controller should generate the control signals was crucial. Two approaches were studied:

- **Positional form:** the controller’s neural network directly predicts the absolute value of the control signal to apply (u_k).
- **Incremental form:** the neural network only predicts the variation of the control signal (Δu_k), and the final action is calculated as $u_k = u_{k-1} + \Delta u_k$.

The **incremental form** was chosen based on its theoretical advantages in the context of neural networks. This approach simplifies learning, as the network only needs to predict small variations, which are usually bounded and have a mean close to zero. This facilitates data normalization, prevents the saturation of activation functions, and reduces the risk of exploding gradients during training. Furthermore, this formulation is consistent with the concept of *residual learning*, which has proven to be more efficient and stable in deep networks, and contributes to a more robust control dynamic by preventing prediction errors from accumulating indefinitely.

```

1 out = controller(inp, training=True)[0]
2
3 mf      += out[0]
4 vtg     += out[1]
5 brk     += out[2]
6 ice_sp  += out[3]

```

Listing 1: Python code for the incremental-form controller.

4. Selection of the controller architecture The controller architecture was defined, opting for a **Stateful LSTM Recurrent Neural Network (RNN) with an MLP (Multilayer Perceptron) prediction head**. This choice is based on the sequential nature of the control problem and the decision to generate incremental actions. The architecture is broken down into four key components:

Stateful LSTM Layer The core of the controller is an LSTM (Long Short-Term Memory) layer configured in **stateful** mode. An RNN was chosen because control problems are inherently time-dependent; the optimal action at instant k not only depends on the current state but also on the sequence of previous states and actions. The LSTM is particularly suitable for capturing these temporal dependencies and complex vehicle dynamics. The **stateful** mode is crucial: it allows the controller to maintain an internal "memory" between discrete time steps. In this way, the controller processes information step by step, just as it would operate in a real environment, remembering its internal state to make more informed and coherent decisions over time.

Normalization of the LSTM output Just after the recurrent layer, a **LayerNormalization** layer is introduced. RNNs, and LSTMs in particular, can be sensitive to the scale of their internal activations, which can lead to instabilities during training (such as vanishing or exploding gradients). Layer normalization stabilizes the learning process by rescaling the activations at each step, resulting in faster and more robust convergence, a fundamental requirement for a reliable control application.

Intermediate dense layer The normalized output of the LSTM feeds into a dense layer with a **tanh** activation function. This layer forms the first part of the MLP head and acts as a nonlinear processing block. Its function is to take the temporal features extracted by the LSTM and learn more complex, higher-level combinations between them. This gives the controller a greater expressive capacity to map the historical and current state of the system to the appropriate control actions, going beyond a simple linear relationship.

Dense output layer The final layer is a dense layer with four neurons and a **linear** activation function. The number of neurons (four) corresponds directly to the four control actions it must generate: $\{\Delta m_f(k), \Delta vtg(k), \Delta brk(k), \Delta ice_sp(k)\}$. The choice of a **linear** activation is a direct consequence of the decision to design an incremental controller. Since the network predicts the **variation** (Δu_k) and not the absolute value, the output must be able to take any real value, positive or negative, without being bounded. The linear activation allows precisely this, delegating the calculation of the final value ($u_k = u_{k-1} + \Delta u_k$) to an operation outside the model, which simplifies the network's learning task. For the complete experimental script associated with this architecture, see Appendix D.

Variants such as non-stateful RNNs, GRUs, and direct outputs without normalization were tested, but the stateful LSTM with LayerNormalization and an incremental MLP head achieved the best overall performance.

5. Trajectory Initialization Strategy Once the incremental controller architecture was defined, an operational challenge arose: how to manage the first step of the simulation ($k = 0$), since by definition there are no previous control actions (u_{k-1}) to start from. The trajectory initialization, therefore, required a specific strategy.

Initially, an approach based on synthetic data generation was proposed. Given that the ICE+PG system models were described as robust to any input within their physical ranges, the first strategy consisted of generating a set of initial actions $\{m_f(0), vtg(0), brk(0), ice_sp(0)\}$ randomly, uniformly sampling within the operational limits of each variable. The hypothesis was that, by executing a first step with

these random actions, the vehicle model would generate a coherent initial state (including speed $v(0)$), allowing the controller to begin its incremental inference cycle from the next step ($k = 1$).

```

1 def sample_init_state():
2     """
3     It returns a tf.Tensor of shape (5,) with random values in the ranges:
4     - vel:      [0,      150]
5     - mf:       [3,      70]
6     - vtg:      [0.23,  0.9]
7     - brk:      [0,      100]
8     - ice_sp:   [0,     4500]
9     """
10    mf      = tf.random.uniform([], minval=3.0,      maxval=70.0,      dtype=tf.float32)
11    vtg     = tf.random.uniform([], minval=0.23,     maxval=0.9,      dtype=tf.float32)
12    brk     = tf.random.uniform([], minval=0.0,      maxval=100.0,    dtype=tf.float32)
13    ice_sp  = tf.random.uniform([], minval=900.0,    maxval=4500.0,   dtype=tf.
14                                float32)
15    state = tf.stack([mf, vtg, brk, ice_sp], axis=0)

```

Listing 2: Sampling of initial state by uniform sampling of the physical ranges of the control actions.

However, in practice, this approach proved to be inadequate. Although each individual value was plausible, the randomly generated combinations lacked the necessary physical correlation, resulting in completely unrealistic initial vehicle states, such as negative speeds or stagnation. After consulting with the developers of the ICE+PG models, it was confirmed that, while the model accepts inputs in broad ranges, these must maintain internal consistency to produce meaningful results. Due to the confidentiality of the exact model dynamics, it was not possible to define a set of rules for generating valid synthetic initial states.

Consequently, a second, more robust and reliable strategy was chosen: sampling the initial states from real trajectory data. The new procedure consists of selecting, at the beginning of each simulation, a random row from a set of data files containing real trajectories. From this row, the values of the control actions $\{m_f, vtg, brk, ice_sp\}$ are extracted, thus guaranteeing their physical coherence as they come from an empirical measurement. This set of actions is then used to execute a single simulation step ($k = 0$) and obtain the complete initial state of the vehicle, primarily the initial speed $v(0)$. Once this starting point is established, from the second time step ($k = 1$), the controller already has a valid initial state and a set of previous actions ($u(0)$), allowing it to begin its incremental operation cycle to predict $\Delta u(1)$.

```

1 def sample_init_state(folder_path="data"):
2     """
3     It returns a tf.Tensor of shape (6,) by taking a random row from a random CSV file
4     in the specified folder and mapping each column to the corresponding position.
5     """
6     csv_files = [f for f in os.listdir(folder_path) if f.endswith(".csv")]
7     if not csv_files:
8         raise FileNotFoundError(f"No CSV files were found in {folder_path}")
9
10    chosen_file = os.path.join(folder_path, random.choice(csv_files))
11
12    # Count how many rows the CSV has (-1 to ignore the header) with open(chosen_file
13    # , "r") as f:
14    n_lines = sum(1 for _ in f) - 1
15    if n_lines <= 0:
16        raise ValueError(f"The file {chosen_file} is empty")
17
18    # Select a random index (0-indexed after the header)
19    random_row_idx = random.randint(0, n_lines - 1)
20
21    # Read only that row
22    df_iter = pd.read_csv(chosen_file, skiprows=lambda x: x != 0 and x-1 !=
23                          random_row_idx)
24    row = df_iter.iloc[0]
25
26    # Extract columns and convert to tensors
27    vtg     = tf.constant(row["vtg"],      dtype=tf.float32)
28    brk     = tf.constant(row["Brake"],    dtype=tf.float32)

```

```

26 ice_sp = tf.constant(row["ICE_Speed_soll"], dtype=tf.float32)
27 EM2    = tf.constant(row["EM2_Torque"], dtype=tf.float32)
28 torque = tf.constant(row["ICE_Torque_pred"], dtype=tf.float32)
29
30 state = tf.stack([vtg, brk, ice_sp, EM2, torque], axis=0)
31 return state

```

Listing 3: Sampling of initial state from real data: random extraction of a CSV row to ensure physical coherence.

6. Controller Update Strategy: Monte Carlo vs. TD Another fundamental decision in the design of the training process was to determine the update frequency of the controller’s parameters. Two main reinforcement learning paradigms were evaluated: updating at the end of the trajectory, characteristic of *Monte Carlo* (MC) methods, and updating at each step, typical of *Temporal Difference Learning* (TD).

Monte Carlo methods update the agent’s policy only at the end of a complete episode, based on the total accumulated return. Their main advantage is that they provide an unbiased value estimate, as they use the complete sequence of actual rewards obtained. Theoretically, this can lead to more stable convergence towards the true optimum.

However, this approach presents a critical disadvantage in the context of our problem: the inefficiency of sampling in long episodes. Given that each simulation trajectory has a duration of 600 seconds with a frequency of 2 Hz (i.e., 1200 time steps), a Monte Carlo method would delay any learning until the completion of each long simulation. This would not only drastically slow down the process but also make credit assignment difficult, that is, identifying which specific actions within a long sequence were responsible for the final outcome. In the initial phases of training, a prohibitive number of complete trajectories would be required for the controller to begin to learn a useful policy.

In contrast, TD-based strategies update the policy at each time step, using the immediate reward and an estimate of the future value. Although this introduces a certain bias (since the update is based on another estimate), it offers an overwhelming computational and efficiency advantage. The controller receives a learning signal at each of the 1200 iterations of the trajectory, allowing for much faster convergence.

Considering the computation time and the length of the simulations, we opted for an incremental update based on temporal difference learning, so that the training cycle would be faster and more practical.

7. Handling of Actuator Saturation via Clipping With the above components defined, the controller training process began. In the first runs, an unstable and physically impossible behavior was observed: certain control signals, such as ‘mf’, diverged to values outside their operational limits.

This phenomenon, while problematic, was a logical consequence of the system’s design. At no point had the neural network been constrained by the physical ranges of the variables it controls. The incremental nature of the controller presented a particular challenge. Directly limiting the magnitude of the increments (Δu_k) that the network predicts is an insufficient solution, since the cumulative sum of small but consistent deltas can also cause a variable’s absolute value to exceed its limits. For example, an actuator with a range of $[0, 100]$ can easily be overshoot if small positive increments are repeatedly applied.

To solve this problem, a saturation technique, commonly known as *clipping*, was implemented. This operation is applied as a post-processing step immediately after the incremental update of the actions and before introducing them into the vehicle model.

The mechanism is direct: after calculating the new value of each control signal, it is compared with its predefined physical limits. If the value exceeds these limits, it is saturated to the corresponding maximum or minimum value. Formally, the final applied action, u_k^{final} , is calculated as:

$$u_k^{\text{final}} = \text{clip}(u_{k-1} + \Delta u_k, u_{\min}, u_{\max})$$

where $u_{\min}$ and $u_{\max}$ are the physical operating limits of the actuator. This procedure was applied to all control signals: $\{m_f, \text{vtg}, \text{brk}, \text{ice_sp}\}$.

The introduction of *clipping* proved to be an effective and computationally economical solution.

```

1 mf += d[0]
2 vtg += d[1]
3 brk += d[2]
4 ice_sp += d[3]
5
6 if clipping:
7     mf = tf.clip_by_value(mf, 3.0, 70.0)
8     vtg = tf.clip_by_value(vtg, 0.23, 0.9)
9     brk = tf.clip_by_value(brk, 0.0, 100.0)
10    ice_sp = tf.clip_by_value(ice_sp, 0.0, 4500.0)
11
12 # Transitions ICE nad PG (differentiable)
13 torque, nox, co = trans_func.predict_ice(ice_sp, mf, T_amb_K, p_amb_bar, vtg)
14
15 if clipping:
16     torque = tf.clip_by_value(torque, -50, 300.0)
17
18 vel_out, soc = trans_func.predict_PG(ice_sp, 0.0, torque, brk)

```

Listing 4: Incremental update of control signals with clipping to ensure actuator saturation within their physical limits.

5.1.2 Model performance improvement strategies

In contrast to the structural decisions presented earlier, which constituted the mandatory pillars for the controller’s operation, this section details a series of optional techniques implemented with the sole objective of improving training performance. The omission of any of these improvements would not prevent the model from working, but their inclusion was key to accelerating convergence, increasing learning stability, and achieving a more efficient final control policy. It should be noted that, because the hyperparameters associated with these improvements are influenced by the inclusion of the improvements themselves, the optimal values of said hyperparameters were determined at the end of the last strategy. That is, when optimizing a hyperparameter and subsequently introducing a new improvement, that value ceased to be optimal in the new context, so it was necessary to adjust all parameters together once all strategies were implemented.

1. Training Stabilization via Gradient Clipping (clipnorm) During the initial phases of training, a notable instability in the loss function was observed, characterized by sharp peaks or ridges. This behavior is a common symptom of exploding gradients, a phenomenon in which an abrupt update of the controller’s weights causes a sudden deterioration in its performance, resulting in a high loss.

To mitigate this problem, the first improvement strategy implemented was gradient clipping. This technique prevents parameter updates from being excessively large and destabilizing the learning process. Specifically, the `[clipnorm=1.0]` argument was used within the Adam optimizer. This mechanism reviews the L2 norm of the gradient at each update step; if it exceeds the threshold (in this case, 1.0), the entire gradient is rescaled so that its norm is equal to that threshold.

```

1 opt = tf.keras.optimizers.Adam(learning_rate, clipnorm=1.0)

```

Listing 5: Adam optimizer configuration with gradient clipping (`clipnorm=1.0`) to prevent exploding gradients during training.

2. Gradual Loss Weighting during Model Warm-up Another phenomenon observed is that, when starting a new simulation, the system starts from a state that may not be representative of the dynamic behavior that is sought to be controlled. The first steps constitute a **transient period** where the internal states of the LSTM models (both the simulator and the controller itself) are adjusting. Penalizing the controller’s errors during this “warm-up” phase can introduce noise into the learning process, as the actions are based on a not-yet-stabilized system state.

To solve this, a strategy of **temporal loss masking** was implemented. The idea is to give the system time to reach a stable dynamic state before evaluating the controller’s performance at its full weight. For this, a weighting factor w was defined that **increases linearly from 0 to 1** during a predefined number of initial steps (`warmup_steps`).

The loss of each step (`step_loss`) is multiplied by this factor w . In this way, the contribution of the errors to the gradient is zero at the beginning of the trajectory and gradually increases, reaching 100% only when the system is considered to be already operating in a stable regime.

```

1 # Calculation of the loss for the current step
2 step_loss = tf.square(vel_target - vel_out)
3
4 # Using the learning rate warm-up (for instance, 200 steps)
5 warmup_steps = 200
6 w = tf.minimum(1.0, tf.cast(i, tf.float32) / tf.cast(warmup_steps, tf.float32))
7
8 # Weighting the loss by factor w
9 step_loss *= w

```

Listing 6: Implementation of gradual loss weighting with a linear warm-up ramp for the first simulation steps.

This technique allows the controller and the simulator to synchronize and reach a coherent dynamic state before the tracking error is fully penalized, promoting more robust training focused on the system’s relevant behavior.

3. Dynamic Warmup Adjustment via Curriculum Learning Although the technique described above allows for stabilizing learning in initial phases, fixing a static value for `warmup_steps` could limit the controller’s performance once it has acquired enough competence. Inspired by the methodology known as *curriculum learning*, a more effective alternative is to dynamically adjust the warm-up period based on the model’s learning level.

In the initial stages, when the controller is still learning and the system is unstable, it is beneficial to maintain a high `warmup_steps` to reduce the negative impact of early errors. However, as the model acquires skill and stability, this period can be gradually reduced. In this way, the controller is progressively forced to improve its performance in the initial situations of the episode as well, increasingly facing more demanding conditions.

This dynamic approach follows a pedagogical principle: initially, the controller is offered favorable conditions (less penalization at the beginning) and later, as its performance improves, the level of demand is gradually increased (fewer warm-up steps). This ensures that training efficiently leverages all available information in each episode, forcing the model to develop robust strategies capable of responding effectively from the early stages of the control process.

In practice, to determine when to reduce the warm-up period, a loss threshold considered favorable was established; once the model managed to reach this threshold taking into account the entire trajectory, the value of `warmup_steps` was reduced, repeating the process until an operational minimum was reached.

4. Dynamic Learning Rate (LR) Reduction via Adaptive Plateau Strategy Despite having previously implemented the gradient clipping technique with `clipnorm=1.0`, a slight residual instability was still observed in the controller optimization process. This behavior suggested the need for additional strategies to ensure more stable convergence and a more efficient exploration of the solution space.

In response to this situation, an adaptive strategy for dynamic learning rate (LR) reduction was implemented, commonly known as **ReduceLROnPlateau**. This technique continuously monitors the evolution of training and reduces the LR value when it detects that the model’s performance has stopped improving for a certain number of consecutive epochs, indicating that the model might be oscillating around a local minimum. Specifically, a tolerance (or *patience*) was established, after which, if no improvement was obtained, the learning rate was multiplied by a factor less than 1 (usually 0.5). Before performing this reduction, the weights of the model that had obtained the best performance

so far were restored, ensuring that the LR reduction always started from the best known point, thus avoiding refining less promising solutions.

In addition, the dynamic warm-up strategy (`warmup_steps`) described earlier interacts with the dynamic LR reduction in a synergistic way. When the model managed to reach a sufficiently low loss (`good_loss`), indicating that it had learned an effective control strategy in the final phase of each episode, the number of warm-up steps was automatically reduced, thus increasing the level of demand on the model. At this point, and to ensure that the model could effectively adapt to the imposed increase in difficulty, the original learning rate value was immediately restored. This LR restoration allowed the model to make significant adjustments in its state-action mapping to adapt to the new loss distribution, maintaining a high learning rate during this transient phase.

5. Early Stopping Based on Patience Even after applying all the previous strategies to stabilize and accelerate training convergence, there was a risk that the process would continue to run unnecessarily for a large number of epochs without significant improvements, leading to unjustified resource consumption or even a deterioration in performance due to overfitting. To avoid this situation, a technique known as **Early Stopping** was implemented, which stops training early when the controller’s performance stops improving for a predefined number of consecutive epochs, called `early_stop_patience`.

During training, in each epoch, the current loss value is compared with the best value obtained so far. If an improvement is obtained, the current configuration of the controller’s weights is saved and a counter called `es_wait` is reset. On the contrary, if no improvement is observed, this counter is incremented by one unit. When the counter reaches the value defined by `early_stop_patience` (usually 50), the training automatically stops, considering that the model has entered a phase in which continuing to train will hardly provide additional benefits.

6. Mitigation of Initialization Sensitivity via Model Screening Finally, it was found that the controller’s final performance exhibited a high sensitivity to the random initialization of its weights. A bad starting point could lead to slower convergence or stagnation in lower-quality local minima, regardless of the optimization strategies applied later.

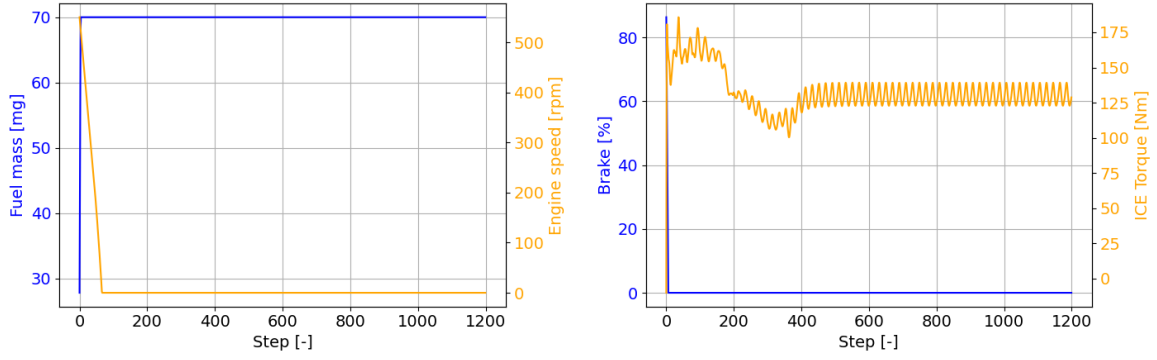
To mitigate this dependence and increase the probability of converging to a high-quality solution, the last improvement strategy consisted of implementing a pre-selection or "screening" phase for models. The procedure is as follows: three instances of the controller are trained starting from three different weight initializations (i.e., with three different random seeds). This initial training is very brief, only five epochs. At the end of this short period, the performance of the three models is evaluated and the one that has achieved the lowest loss function is selected.

Only this "promising" instance is kept to continue with the complete training process, discarding the other two. This technique, although computationally simple, effectively increases the robustness of the training, ensuring that the main computational effort is invested in a learning trajectory with a greater potential for success from the beginning.

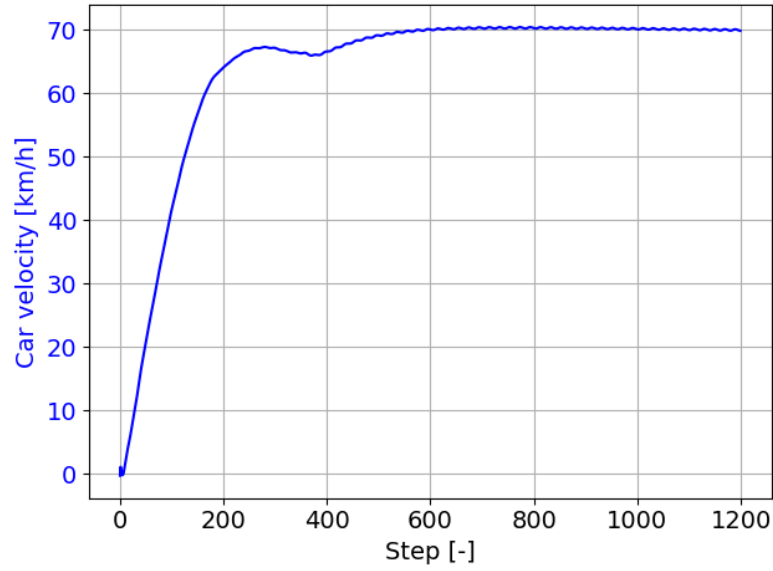
5.1.3 Controller Results with V1 Models

Up to this point, analyzing the controller’s results was a futile effort, since without the techniques described in the previous section, the performance was very poor. Once all the "Model performance improvement strategies" were applied, the code for which can be found in Appendix E, the controller achieved a stability that allowed, for the first time, a formal validation of its behavior.

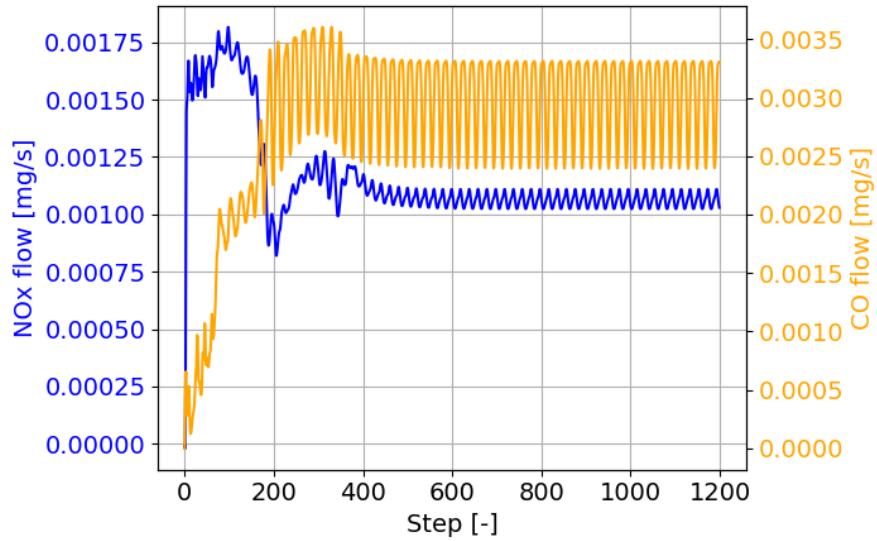
The validation method was direct: after completing the training cycle, the controller with the best performance was selected. With this final model, a complete simulation trajectory was generated to visually observe the behavior of all key variables. Although the initial objective was only to test a cruise speed of 70 km/h (Figure 6), it was decided to test the robustness of the controller architecture in other scenarios, adjusting the hyperparameters for speeds of 10 km/h and 130 km/h (Figures 7 and 8).



(a) Model inputs (70 km/h)

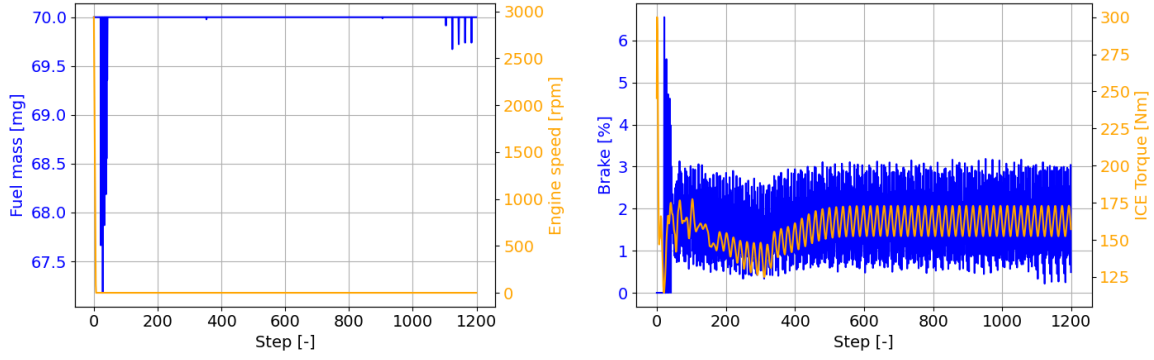


(b) Velocity outputs (70 km/h)

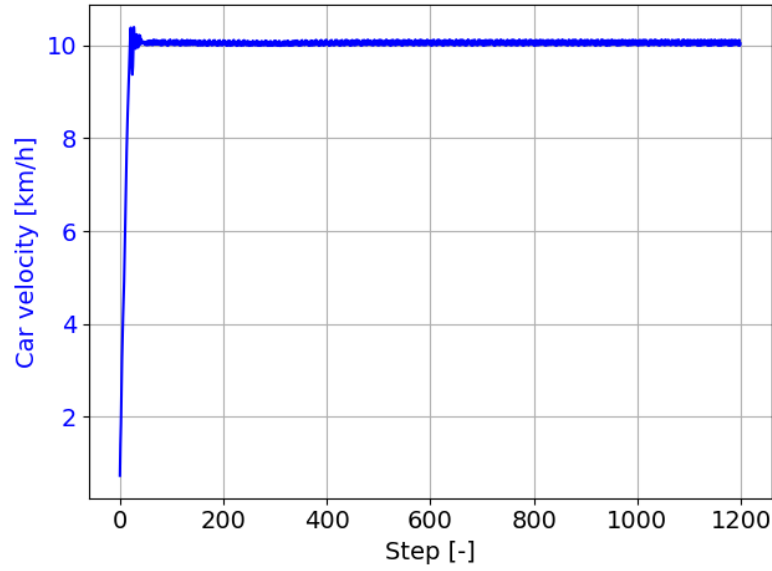


(c) Pollutant outputs (70 km/h)

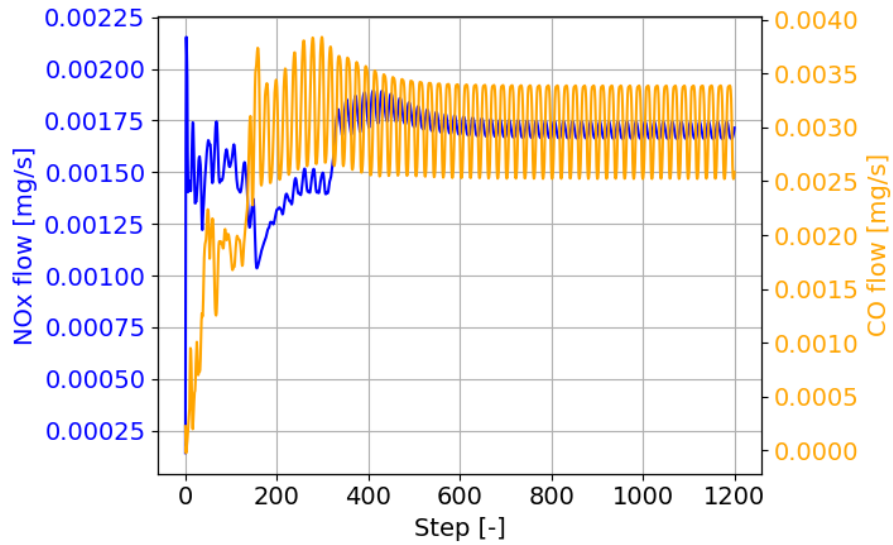
Figure 6: Controller behaviour with V1 models at a target speed of 70 km/h.



(a) Model inputs (10 km/h)

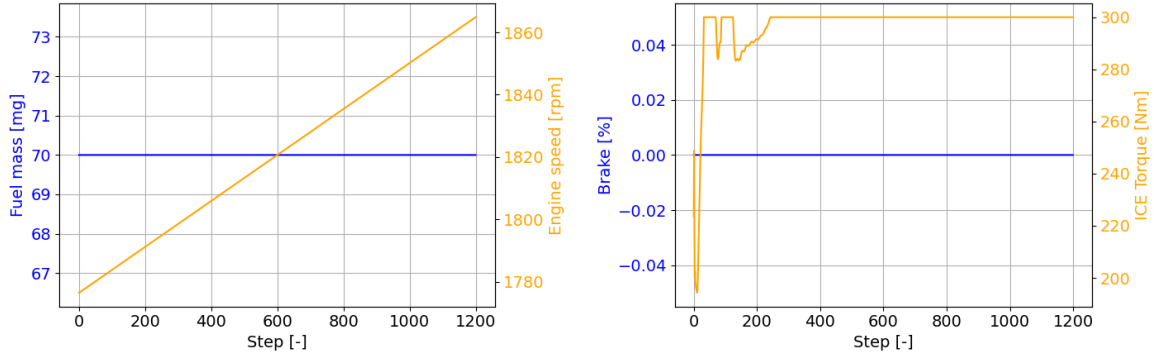


(b) Velocity outputs (10 km/h)

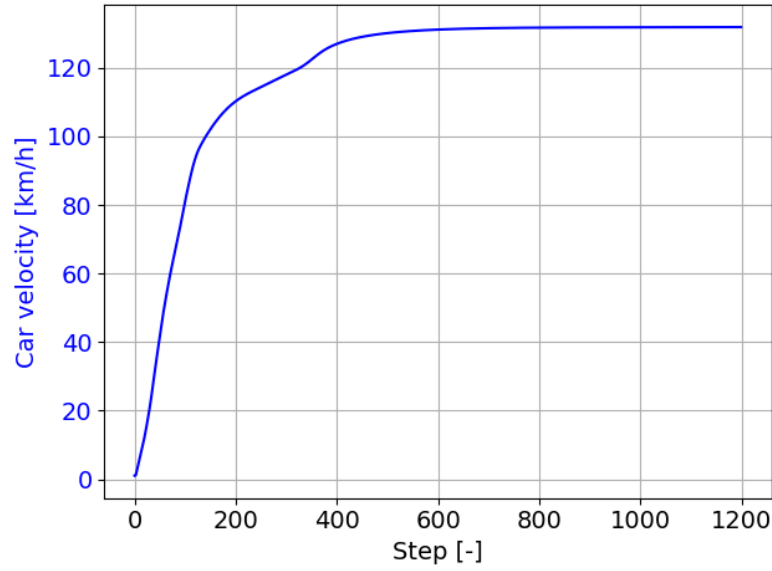


(c) Pollutant outputs (10 km/h)

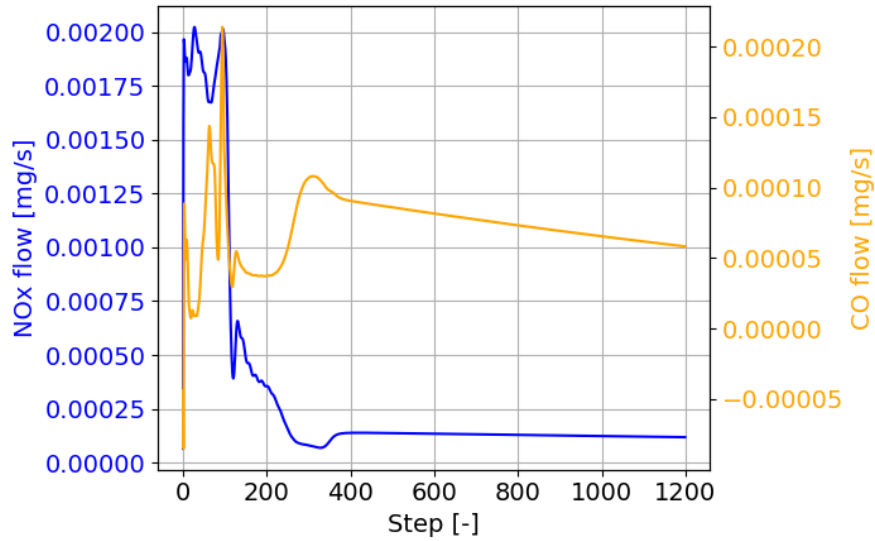
Figure 7: Controller behaviour with V1 models at a target speed of 10 km/h.



(a) Model inputs (130 km/h)



(b) Velocity outputs (130 km/h)



(c) Gas outputs (130 km/h)

Figure 8: Controller behaviour with V1 models at a target speed of 130 km/h.

Analysis and subsequent invalidation of the results At first glance, the results could be considered moderately satisfactory. The controller managed to follow the reference speed, although a clear difference in stability could be seen, with the model finding it much easier to stabilize at low speeds than at high speeds.

However, upon presenting these findings to the ICE+PG model development team, **the results were invalidated**. We were informed of two critical factors that we had not taken into account:

1. **ICE model restriction:** We had not been previously informed of the fact that the combustion engine model is only valid for `ice_sp` $\geq$ 900 rpm (a condition introduced in **V2** of the history). Our simulations did not respect this restriction, therefore generating physically incorrect behaviors.
2. **New versions of the models:** We were strongly advised to use the most recent and improved versions of the models, specifically **V2.1** for the ICE and **V2.3** for the PG, which offered superior performance and corrected certain behaviors of the initial version.

Therefore, although this exercise served as a valuable proof of concept for the controller architecture and training strategies, the numerical results could not be considered representative of the real system. The process of adapting to these new conditions and the final results obtained with the correct system configuration are described in detail in the following section.

5.2 Second problem: combustion-only vehicle (cruise control) with V2 models

After validating the controller architecture and the training methodology in the previous phase, the next step was to apply this design to the correct scenario, using the most recent and precise versions of the simulation models. The control objective remained identical: to develop a cruise controller for a combustion-only vehicle.

5.2.1 Controller Modifications

The transition to V2 models and the incorporation of new operational restrictions did not require a complete redesign, demonstrating the modularity and robustness of the methodological approach developed in Phase 1. However, it was necessary to adapt specific controller components to ensure compatibility and compliance with the new conditions. These modifications are detailed below.

1. Update of the Trajectory Initialization Strategy The elimination of the variable geometry turbocharger (`vtg`) as a control variable in the ICE V2.1 model required a direct adaptation in the strategy used to initialize each simulation episode.

The `sample_init_state` function, responsible for sampling a physically coherent starting point from real trajectory data, was updated to align with the new data format and the new model interfaces. Specifically, the implementation was modified so that the `vtg` value is no longer extracted from the CSV files, thus ensuring compatibility with the new data trace format and providing the system with a coherent initial state for the V2 models.

2. Adaptation of the controller to physical constraints (`ice_sp` $<$ 900) The change from V1 to V2 of the models was not limited only to the elimination of the VTG signal, but also revealed a critical operational restriction previously not considered: if the combustion engine speed signal (`ice_sp`) is less than 900 rpm, then the engine does not produce torque, so the signals associated with the engine (`mf` and `torque`) must be strictly zero.

Initially, a simple approach would be to modify these controller outputs with an external condition (e.g., a conventional `if` in Python). However, this would create a serious problem during controller training, because the signal corrected by the manual code would no longer correspond to the signal the neural network had actually calculated. In other words, if the controller were to predict, for example, an engine speed of 400 rpm with a positive fuel value (`mf`=30), artificially correcting that value to 0 would break the gradient continuity, preventing the controller from correctly learning this physical restriction.

The ideal solution is for the neural architecture itself to learn this relationship, being able to directly infer that if the prediction for `ice_sp` falls below the 900 rpm threshold, then the associated fuel output should naturally tend toward zero. However, this presents an additional challenge due to the original incremental design of the controller, in which the network only predicts incremental variations and not absolute values. By working exclusively with increments, the network is unaware of the real absolute value of the signals, which makes it impossible to directly impose restrictions based on the absolute value of a signal like `ice_sp`.

To simultaneously solve these two problems, a fundamental modification of the controller was proposed and developed, based on four clearly defined principles:

1. **Incremental model with internal absolute output**

The architecture must maintain the incremental philosophy because of its proven advantages in stability and robust learning, but the conversion of the predicted increment to the absolute value will be done within the neural network itself, rather than externally. This implies that, although the network is internally operating with increments, its direct output is already a final absolute value.

2. **Controllable output using soft activation layers**

Until now, actuator saturation was managed by applying hard clipping to the final values. This strategy, although effective in physical terms, has significant limitations from a learning perspective: the abrupt clipping blocks the propagation of gradients, preventing the network from detecting when it is trying to exceed a physical limit and hindering convergence. For this reason, hard clipping was replaced by a soft solution using sigmoidal functions. Specifically, the network generates a "raw" absolute prediction (`abs_raw`), which is then smoothly transformed to the allowed physical range using the following formula:

$$\text{bounded} = \text{lower} + (\text{upper} - \text{lower}) \cdot \sigma(\text{abs_raw})$$

where $\sigma(x)$ is the standard sigmoid function. This strategy ensures that the output values always remain within their physical limits, and simultaneously allows for smooth gradient propagation, making it easier for the neural network to learn to respect those limits.

3. **Differentiable implementation of the threshold for `ice_sp` < 900 rpm**

Finally, to impose the specific restriction that when `ice_sp` < 900 rpm, the `mf` signals must be zero, an additional soft activation strategy (a gate) was used. This soft gate is activated when the predicted `ice_sp` signal exceeds the physical threshold (900 rpm). In detail, a scaled parameter α is defined, which controls the harshness of the transition, and a gate defined by the sigmoid function in the form:

$$\text{gate} = \sigma(\alpha \cdot (\text{ice_sp} - 900))$$

This gate generates values close to zero when `ice_sp` is below the threshold and close to one when it exceeds it. By applying this gate multiplicatively to the fuel signal (`mf`), the network naturally learns to deactivate that signal when `ice_sp` is physically insufficient to produce torque.

4. **Consistent and specific scaling to facilitate training**

Throughout the development of the controller, it was detected that, even with the previous principles implemented, the model was unable to train correctly when the variables were used directly in their original physical space. The reason lies in the enormous differences in magnitude between the input and output variables (for example, `mf` in tens versus `ice_sp` in thousands), which generates numerical instability and hinders gradient flow in neural networks. To address this problem, an explicit scaling was incorporated both in the inputs and outputs of the model, and also in the internal physical ranges and thresholds used in the soft activation functions. This scaling ensures that all the variables the network works with are in a comparable and well-conditioned range for training, typically close to $[-1, 1]$, which stabilizes optimization and allows physical constraints to be applied coherently in the same representation space that the network uses. Instead of designing an arbitrary scaling, it was decided to reuse the existing and validated scalers in the ICE and PG models, which had already been trained on real data and therefore contained a normalization consistent with the physical distribution of the variables. However, the ICE and PG scalers include many more variables than are necessary for the controller.

Therefore, a specific tool (**ScalerLibBuilder**) was developed, capable of building a new custom scaler, extracting only the relevant columns from the original scalers and assembling them into a single object. This tool allowed for the creation of a "collage" of the required scalings, ensuring that the controller works in the same reference system as the rest of the ecosystem, without introducing unnecessary variables or inconsistencies. The detailed implementation of this tool can be found in Appendix G.

It is important to mention that the core of the architecture, a **Stateful LSTM Recurrent Neural Network (RNN) with an MLP prediction head**, remains unchanged. The principles described above correspond to pre- and post-processing operations that wrap this core, adding the ability to handle physical constraints and improving numerical stability without altering the base prediction model.

For a detailed description of the complete implementation of the controller adapted to the constraints of the V2 models, as well as its source code, please refer to Appendix F.

5.2.2 Controller Results with V2.2 Models

Once the modifications were implemented to adapt the architecture to the new constraints and the V2 models, the full implementation of which is detailed in Appendix H, the final validation of the controller was carried out. The method was analogous to that of the previous phase: after training, the model with the best performance was selected and a complete simulation trajectory was generated to analyze its behavior in different scenarios. On this occasion, the results are presented for speeds of 70 km/h (Figure 9) and 20 km/h (Figure 10).

Analysis of results The results obtained with the architecture adapted to the V2 models demonstrate a significant advance. As can be seen in the figures, **all models correctly manage to stabilize** at their respective target speeds, fulfilling the fundamental purpose of the controller.

A detailed analysis of the graphs confirms that the controller consistently respects the imposed physical constraints. It can be observed that the control signals remain within the specified ranges and, crucially, that the condition that the engine torque is zero if `ice_sp` is less than 900 rpm is effectively met. Furthermore, the **stabilization is notably faster and more precise** than in the tests with V1 models, which validates the effectiveness of the implemented modifications, such as the differentiable management of constraints and the coherent scaling of variables.

Even so, it should be noted that some limitations persist. A stable model could not be optimized for very high speeds, and the controller's behavior still exhibits a certain degree of instability. Additionally, the challenge of generating initial trajectories with greater physical coherence remains unresolved.

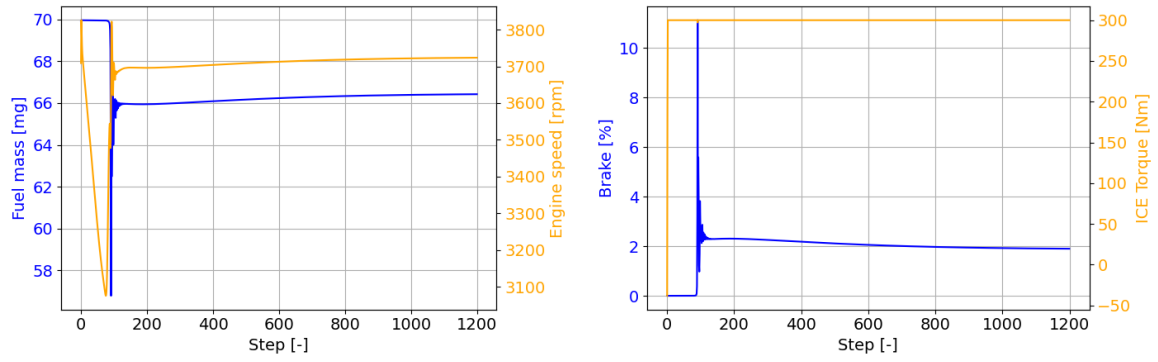
These observations indicate that the current architecture, based exclusively on a supervised neural network, might be approaching its performance limit. The improvements obtained are clear, but the residual problems suggest that to achieve more robust and optimal control throughout the entire operating range, it might be necessary to explore more advanced paradigms, such as reinforcement learning.

Knowledge Transferable to the Design of an RL Agent Although the current model is limited, this work establishes an indispensable methodological foundation and allows for the extraction of key lessons that will be directly reused in the design of a future Reinforcement Learning agent. The knowledge acquired allows us to state that a successful RL model for this problem should incorporate the following principles:

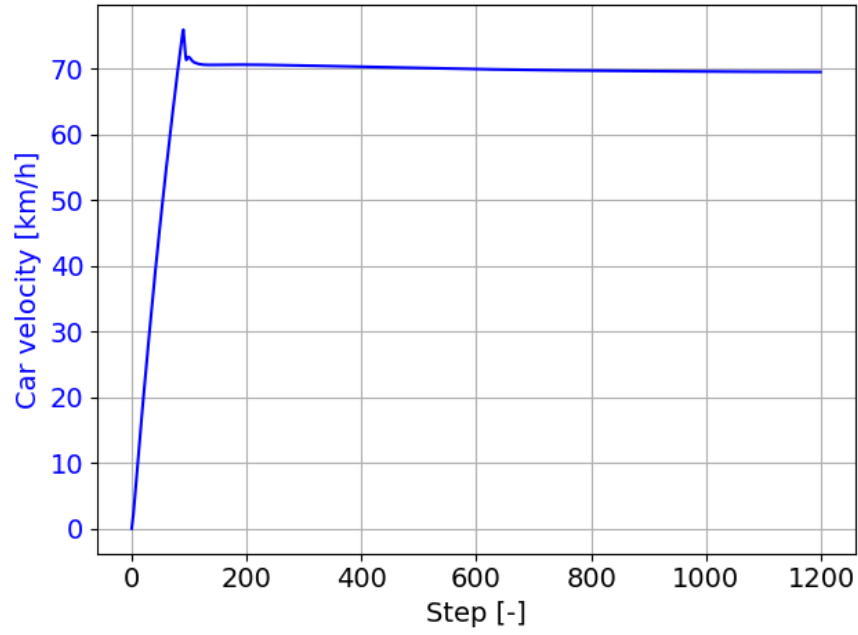
1. **Base architecture:** The choice of a **stateful recurrent (LSTM) network** has proven to be fundamental for processing the sequential nature of the control problem.
2. **Controller format:** The design of an **incremental controller** that predicts variations instead of absolute values is a robust strategy that facilitates learning.
3. **Handling of physical constraints:** It is crucial that the gradient is **smooth and continuous**. The implementation of physical constraints using soft activation functions (sigmoid gates) instead of hard clipping is a requirement for the agent to learn the operational limits naturally.

4. **Learning strategy: Curriculum Learning**, dynamically adjusting the difficulty of the problem (such as the weighting of the loss in initial phases), is indispensable for guiding the agent towards a competent control policy.
5. **Training stabilization**: It is necessary to jointly apply methods to stabilize the loss function, such as gradient clipping (`clipnorm`) and **dynamic learning rate reduction** strategies, to prevent instability and ensure convergence.
6. **Coherent initialization**: The strategy of sampling initial states from **real trajectory data** must be maintained to ensure that the agent starts each episode in a physically plausible state.

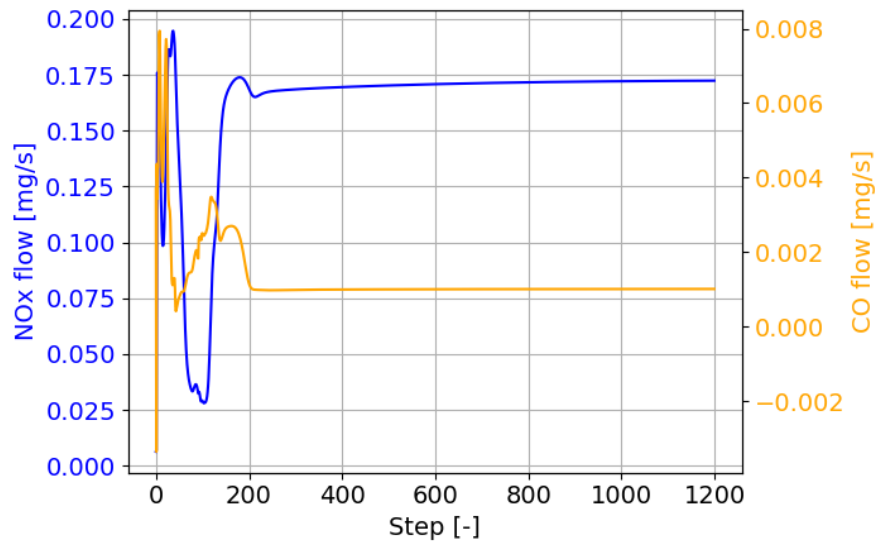
These empirically validated pillars will serve as a starting point to build an RL agent with a higher probability of overcoming the limitations of the current approach.



(a) Model inputs (70 km/h)

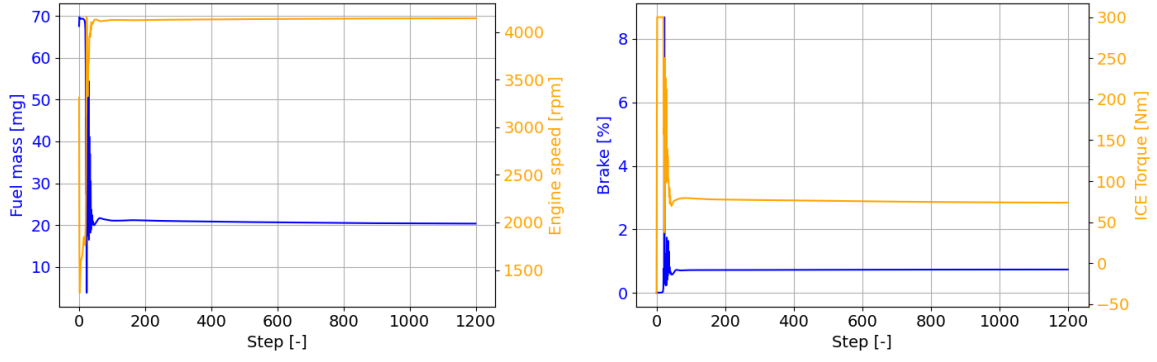


(b) Velocity outputs (70 km/h)

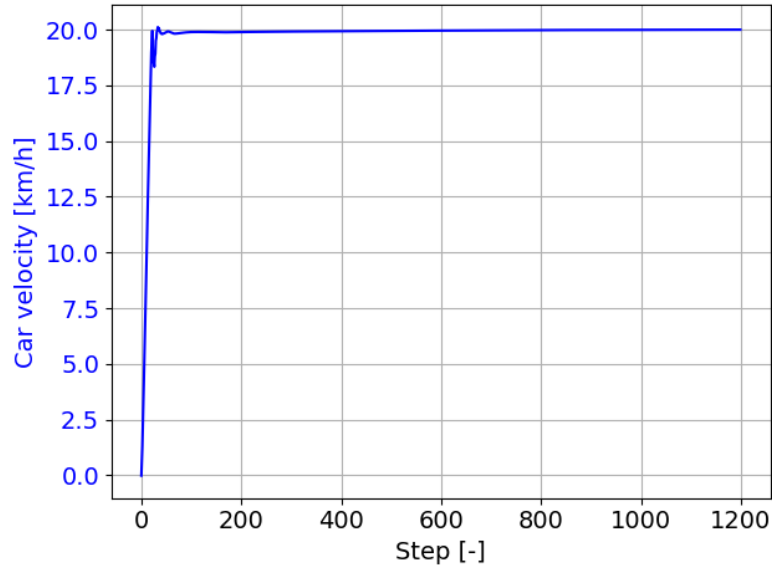


(c) Gas outputs (70 km/h)

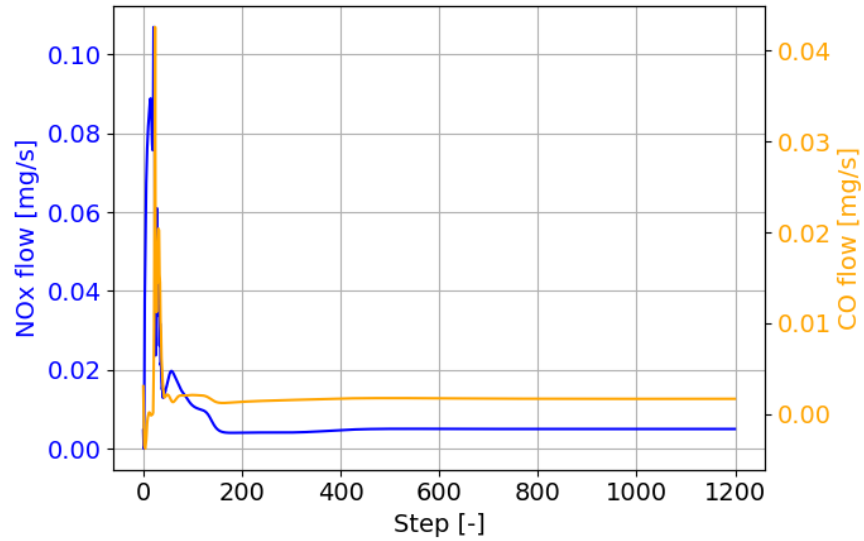
Figure 9: Controller behaviour with V2 models at a target speed of 70 km/h.



(a) Model inputs (20 km/h)



(b) Velocity outputs (20 km/h)



(c) Pollutant outputs (20 km/h)

Figure 10: Controller behaviour with V2 models at a target speed of 20 km/h.

6 Objective and Paradigm Shift (from “Gradients-based” to RL)

The architecture of the simulation environment, based on two LSTM networks (PG and ICE), introduces a fundamental peculiarity in the formulation of the control problem. These networks maintain internal memory states, or “contexts,” which are crucial for generating the system’s temporal dynamics. However, these states do not correspond to physical quantities measurable in a real vehicle, but are artifacts inherent to the simulator model. This discrepancy between the complete state of the simulation and the variables observable in reality is the root of the challenge.

In sequential decision theory, a problem is modeled as a Markov Decision Process (MDP) only if the state the agent receives at each instant is sufficient to encapsulate all relevant past information. In our case, the observations available to the agent are limited to physical variables (speeds, setpoints, etc.), which by themselves do not capture the complete state of the simulator (which includes the hidden contexts of the LSTMs). Consequently, the Markov property is not met from the agent’s perspective, and the problem must be treated as a Partially Observable Markov Decision Process (POMDP). Artificially exposing the simulator’s internal states to the agent would turn the problem into an MDP, but at the cost of creating a spurious dependency that would prevent its application in a real system.

How to Handle a POMDP in Practice: Recurrence in the Agent

To operate with partial observability without depending on the simulator’s internal variables, the agent must build its *own temporal context*. We introduce an internal latent state z_t that summarizes the history and is updated deterministically:

$$z_t = f_\psi(z_{t-1}, o_t, a_{t-1}),$$

where f_ψ is a recurrent function (e.g., LSTM or GRU). The deterministic policy produces the action from that representation,

$$a_t = \pi_\theta(z_t),$$

and the critics learn functions $Q_{\phi_i}(z_t, a_t)$ to guide temporal difference learning. This design concentrates memory where it should be (in the agent), avoids overfitting to simulator artifacts, and maintains consistency between training and deployment: the set of observations that feeds decision-making is the same in both contexts.

This formulation conditions the training procedure. Recurrent networks cannot be trained with isolated transitions; *windows* of sequences are required. In each mini-sequence, a *burn-in* (several steps without gradient) is applied to warm up the hidden state and an *unroll* (steps with gradient) to update parameters. Furthermore, the target networks must replicate the same latent dynamics so that the bootstrapping objectives are consistent.

6.1 Recurrence Architectures and Model Choice

The introduction of recurrence in an Actor-Critic framework can be approached with different architectures. Below, the two main strategies are presented:

Separate Recurrent Actor and Critics. In this configuration, each neural network operates with complete independence. The actor maintains its own hidden state z_t^{act} (from observations and previous actions) and each critic maintains its own, $z_t^{Q_1}$ and $z_t^{Q_2}$. Internal representations of the history are not shared. Consequently, the target networks (π^-, Q_1^-, Q_2^-) must replicate this same structure to calculate the TD targets with their *own* memory, which is updated smoothly via Polyak averaging.

Recurrent Actor with Dependent Critics. In this alternative, computational efficiency is prioritized by sharing a single representation of the history. The actor’s policy builds a hidden state z_t^{act} , and the critics evaluate the pair (z_t^{act}, a_t) . It is a common practice to apply the *detach* operation on z_t^{act} before passing it to the critics, to prevent the gradient of the latter from flowing to the actor’s parameters through the shared memory state.

Justification for the Adopted Choice: Separate Recurrent Actor and Critics

In this task, the first architecture is chosen, providing independent memory to the actor and the two critics. This decision is based on several key advantages in terms of stability and flexibility. Mainly, the separation of memories promotes **greater stability**, as the learning objectives calculated by the *target* networks are less sensitive to changes in the actor’s policy and, furthermore, the critic’s memory is not “contaminated” by the stochastic noise added for exploration. This independence also prevents the formation of **representation bottlenecks**, allowing critics to model temporal dependencies crucial for value estimation even if they are irrelevant to the policy. Finally, at a practical level, this architecture offers remarkable **simplicity in implementation**, as the training procedure is conceptually homogeneous for all networks, which facilitates debugging and system reproducibility.

7 RL Model: Implications and Conceptual Design

A reinforcement learning model is composed of four elements that work in a cycle: an *agent* that decides, an *environment* that responds, an *action space* that the agent can apply, and a *reward* that quantifies performance. In this task, these elements are precisely defined on the ICE-PG system to obtain a clear contract between what the agent can observe, how it acts, and how its behavior is evaluated.

Agent. The agent is the brain of the control system, the entity that learns and makes decisions. Its fundamental purpose is to develop an action strategy, called a *policy*, that allows it to interact with the environment in the most effective way possible to maximize a long-term reward signal. At each time step, the agent perceives the system’s state through a set of observations. Since our problem is partially observable, it does not limit itself to reacting to instantaneous information, but rather integrates these observations with its internal memory to build a contextual understanding of the situation. Based on this belief state, its policy calculates the control action it considers optimal.

Environment. It encapsulates the hybrid vehicle’s dynamics using the same IGE and PG models from the *model-based* phase (`transition_function_model`), but now in *forward-only* mode, unlike the first part of the project: no backpropagation is performed through the simulator. Likewise, due to the previous reason and the fact that RL models have been standardized to the PyTorch library, the TensorFlow code from the first part has been adapted to coexist with the RL loop in PyTorch without the need to propagate the gradient.

The environment represents the universe with which the agent interacts; it is the materialization of the hybrid vehicle’s dynamics and the rules of the problem. For this task, said dynamics are encapsulated by reusing the same *transition_function_model* class developed in the *model-based* phase, thus guaranteeing coherence between both stages.

However, its role in the RL paradigm is fundamentally different. Here, the environment operates as a “black box” in *forward-only* mode: it receives an action from the agent, updates its internal state, and returns a new observation along with a reward signal, without the agent having access to its internal workings. Unlike the first phase, no backpropagation is performed through the simulator and the standardization of PyTorch for RL models. This conceptual and technical separation motivated the adaptation of the original TensorFlow code to interact agnostically with the RL loop, implemented in PyTorch. In this way, communication between the agent and the environment is limited to the exchange of data (actions, observations, rewards), without the need to propagate gradients between the different *frameworks*, providing a new version of said class.

Actions. The actions represent the control decisions that the agent executes on the environment. To ensure coherence with the first phase of the task, it is defined that the RL agent’s action space is identical to the outputs generated by the *model-based* controller. This implies inheriting the crucial design decision of whether the policy should generate absolute values (e.g., “fix the engine at 2000 rpm”) or incremental ones (e.g., “increase by 100 rpm”). Although the *model-based* phase already opted for an absolute approach, this choice becomes even more critical in RL due to exploration strategies.

Standard exploration methods, such as adding Gaussian noise, operate much more stably and predictably on an absolute target value. Applying noise to an incremental action is inherently problematic: the same level of noise could be insignificant for a large desired change, but destabilizing for a fine adjustment. To ensure compatibility with these fundamental RL techniques, it is defined that the agent’s policy always emits absolute setpoints for the control trio (mf , brk , ice_sp): fuel mass (mg), brake percentage (%) and engine speed (rpm).

To guarantee the integrity and physical coherence of the simulator, the environment implements a *safety layer* that processes the agent’s actions. This layer is responsible for rescaling the policy’s outputs (which operate in a normalized range of $[-1, 1]$) to their physical units, and then applying a saturation to their valid ranges:

$$mf \in [3, 70] \text{ mg}, \quad brk \in [0, 100] \%, \quad ice_sp \in [0, 4500] \text{ rpm}$$

Additionally, a physical gate is applied that nullifies fuel injection ($mf = 0$) if the engine speed is below 900 rpm. This logic protects the system from physically incoherent commands.

Observations. Observations are the set of signals that the environment provides to the agent at each time step, representing its only sensory window into the system’s state. It is crucial to highlight that, as previously discussed, this set is deliberately limited to measurable physical quantities. It does not include any internal simulator variables, such as the LSTM context states, consolidating the problem’s formulation as a POMDP. Specifically, the observation vector that the agent receives is:

$$o_t = (v_{\text{ref}}, v, mf, brk, ice_sp)$$

This set of variables is, by design, identical to the input vector used by the controller in the *model-based* phase, ensuring that both approaches solve the problem using the same available information. These variables are kept in their physical units and are externally normalized before being processed by the agent’s neural network.

Reward. The reward function is the numerical signal that guides the entire learning process. Its purpose is to quantify how desirable the current state of the system is as a result of the agent’s previous actions. The fundamental objective of the RL algorithm is to discover a policy that maximizes the accumulated sum of this reward over time. For this task, the reward translates the vehicle’s control criteria into this numerical language. In its main implementation, it focuses on the precise tracking of the reference speed:

$$r_t = -\alpha (v_{\text{ref}}(t) - v(t))^2$$

where $\alpha > 0$ is a hyperparameter that adjusts the scale of the penalty for the quadratic error. The environment’s design allows this function to be easily extended to include additional terms that penalize emissions (β) or control jerkiness (γ), thus maintaining coherence with the cost function of the *model-based* phase.

Environment step cycle. Upon receiving the agent’s action, the environment: (i) rescales and saturates (mf, brk, ice_sp); (ii) applies the ICE gate below 900 rpm; (iii) predicts torque and emissions with `predict_ice`; (iv) predicts speed with `predict_PG`; (v) builds the new observable state; (vi) calculates the reward according to the speed error; and (vii) evaluates termination by stability (speed within a band for n_{stable} consecutive steps) or truncation upon reaching `length_problem`. The `reset()` cleans the simulator’s internal states and samples physically coherent initial conditions from real data before computing the initial speed.

The dialogue between the agent and the environment is articulated through the `step` function, which executes a complete interaction cycle for a single time step. The process follows three logical phases.

First, the environment **processes and interprets the action** received from the agent. This initial phase consists of translating the action from the agent’s normalized space to its corresponding physical units and applying a safety layer that guarantees the simulator’s coherence and protection through saturations and physical rules (such as the nullification of fuel at low revolutions).

Next, the **state transition is simulated**. With the action already validated, the environment uses its internal models (ICE and PG) to calculate the evolution of the vehicle’s dynamics and determine the new state of the system.

Finally, the environment **generates the feedback** for the agent. It builds the new observation vector, calculates the reward that quantifies performance on the task, and evaluates whether the episode has concluded, either by reaching a terminal state (e.g., maintaining a stable speed) or by reaching the maximum temporal horizon.

The restart of this cycle, managed by the `reset()` function, is responsible for initializing each new episode, cleaning the internal simulation states and establishing realistic initial conditions sampled from real driving data.

Explicit definition of the POMDP in this task. Under the restriction of using only physical observables, ICE-PG control is modeled as a POMDP $(\mathcal{S}, \mathcal{A}, \mathcal{O}, P, \Omega, R, \gamma)$:

- **True state** ($s_t \in \mathcal{S}$): Not observable by the agent; includes the complete internal dynamics of the ICE and PG models.
- **Action** ($a_t \in \mathcal{A}$): Corresponds to (mf, brk, ice_sp) , subject to physical limits and logical gates.
- **Observation** ($o_t \in \mathcal{O}$): The information available to the agent, $(v_{ref}, v, mf, brk, ice_sp)$, is a deterministic function of the state.
- **Transition** ($s_{t+1} \sim P(\cdot | s_t, a_t)$): Induced by the `transition_function_model` model, provides the dynamics of the system.
- **Reward** (r_t): Defined as the negative of the squared speed error, $R(s_t, a_t, o_t)$, although it is extensible.
- **Discount factor** ($\gamma \in (0, 1)$): Weighs the importance of future rewards to optimize the long-term return.

With this formulation, the agent assumes the responsibility of adding temporal information (internal memory) to mitigate the partiality of o_t ; the environment guarantees physical consistency and a stable API; and the reward maintains continuity with the original controller objective. The result is a clear and reproducible conceptual design that delimits what the agent learns, how it interacts with ICE-PG and according to what criterion its performance is evaluated.

8 Selected Algorithm: Recurrent TD3 (justification and configuration)

The problem to be solved is to stabilize the vehicle at a fixed speed setpoint while respecting physical constraints and minimizing unwanted effects (emissions, harshness). The formulation is a POMDP: the agent does not observe the system’s complete state because the internal states of the ICE and PG LSTMs are not measurable quantities; therefore, the policy must incorporate memory to integrate recent temporal information. Consequently, the solution requires a *recurrent* agent capable of building a latent context and deciding on continuous actions with precision and stability.

In this framework, there are three reasonable options:

- *Recurrent SAC*: offers a stochastic policy with entropy maximization that favors exploration and is robust off-policy; it can be advantageous when the solution landscape contains multiple local optima, although residual stochasticity can introduce *jitter* in fine tracking.
- *Recurrent PPO*: stands out for its stability and training simplicity with recurrent policies and values; however, being *on-policy*, it needs large amounts of fresh experience and makes poor use of stored data, which is inconvenient when simulating ICE-PG is costly.
- *Recurrent TD3*: uses a deterministic policy and two twin critics, with specific mechanisms to stabilize learning in continuous control; being *off-policy*, it efficiently reuses experience and, in evaluation, delivers noise-free actions, which is especially suitable for stabilizing a *setpoint*.

Recurrent TD3 is chosen because it combines three properties aligned with our objective: a deterministic policy that facilitates clean setpoint tracking; sample efficiency from being off-policy; and

stability mechanisms (twin critics and target action smoothing) well-suited for continuous control under saturations.

What TD3 implies and requires in our case. The policy must be *recurrent*: the actor maintains an internal state built from the recent sequence of observations (and, if applicable, previous actions) to compensate for the environment’s partiality. The critics also incorporate memory, independently of the actor, to evaluate actions with their own stable temporal representations. The **actions** produced by the actor are generated in a $[-1, 1]$ space (via a **tanh**-type activation) and then *re-scaled* to the physical domain of the environment. This normalization is a practical requirement in TD3: the entire *pipeline* of exploration and smoothing assumes bounded actions.

During *training*, TD3 adds **Gaussian noise** to the policy’s actions for exploration (zero-mean, moderate-variance noise, clipped to the $[-1, 1]$ range before physical re-scaling). Additionally, it applies *target policy smoothing* when calculating value targets: the target action used by the critics is perturbed with a small Gaussian noise and re-clipped, which prevents the critics from latching onto non-smoothed peaks of the Q function. In *evaluation*, the policy is **deterministic and non-stochastic**: no noise is injected, thus prioritizing precise stabilization of the target value.

The algorithm maintains **two independent critics** and, in each update, uses the minimum of their estimates to reduce the systematic overestimation of Q typical of actor-critic methods with an approximate value function. The policy is **updated with a delay** relative to the critics, so the actor is trained against more stable value estimators. **Target networks** are used for both the actor and critics, updated by soft averaging (*Polyak*), which dampens the drift of temporal targets. As an off-policy method, TD3 learns from a replay buffer that allows for data reuse and decoupling of collection and learning. However, the introduction of memory into the agent forces us to redefine its structure. A conventional buffer stores transitions (o_t, a_t, r_t, o_{t+1}) independently. If we were to sample in this way, a recurrent network could receive a sequence of steps belonging to completely different trajectories, generating an incoherent temporal context and faulty learning. To avoid this, the buffer is designed to store complete trajectories. In this way, when sampling a *batch* for training, sequences of steps are extracted from these trajectories.

9 Architectures: Actor RNN and Critics RNN

The design of the networks pursues three technical and verifiable objectives: (i) to precisely stabilize the speed setpoint, (ii) to operate under partial observability without relying on internal simulator artifacts, and (iii) to output continuous actions that inherently respect physical ranges. With these preconditions, the policy (actor) and the value functions (critics) are formulated as *recurrent* models and are structured around a clear *range contract*: inputs normalized to $[-1, 1]$, policy outputs in $[-1, 1]^3$, and subsequent rescaling in the environment to physical magnitudes with a safety layer.

Functionally, the actor retains the same *signature* as in the *model-based* approach: same physical inputs $(v_{\text{ref}}, v, mf, brk, ice_{\text{sp}})$, memory for temporal dependence, and three control channels. The key difference now is not *what* it calculates, but *how* it guarantees that the entire pipeline respects ranges and constraints with well-conditioned differentiability. Specifically: coherent normalization with the environment, incremental summation with final clipping, and a differentiable physical gate for fuel.

9.1 Actor RNN

Purpose. To implement a deterministic policy with memory that translates the normalized physical observation into a normalized, bounded action, which is stable by design and compatible with the environment.

Architecture.

Input Normalization. The physical observation is brought to $[-1, 1]$ with the *same* scaling parameters used by the simulator. This symmetry avoids mismatches between what the network sees and what the environment executes.

Temporal Feature Extraction. An LSTM captures the dynamics; its output is stabilized with *LayerNorm*. This reduces sensitivity to magnitudes and improves the regularity of the activation flow.

Incremental Head. An intermediate linear block (without activation) transforms the temporal representation, followed by a head with tanh that produces $\Delta u \in [-1, 1]^3$ (normalized increments for mf, brk, ice_{sp}). Working with increments dampens oscillations and facilitates fine adjustments around the setpoint.

Residual Sum and Final Clipping. The absolute proposal in the policy space is obtained by summing Δu to (mf, brk, ice_{sp}) from the normalized observation. As this sum can fall outside of $[-1, 1]$, a *final* tanh is applied to return the action to $[-1, 1]$. This step is the ultimate guarantor of the *range contract*.

Differentiable Physical Gate for mf . To reflect that if $ice_{sp} < 900$ rpm no fuel should be injected, a soft gate is introduced in the normalized space. Let τ be the equivalent threshold to 900 rpm after normalization; it is defined as:

$$g = \sigma(\alpha(ice_{sp} - \tau)), \quad mf_{final} = g \cdot mf_{proposto} + (1 - g) \cdot (-1)$$

where α adjusts the transition’s harshness and -1 is the normalized minimum (no injection). Thus, the policy learns the physical rule without introducing discontinuities; the environment, already in real units, also applies its hard validation and saturations.

Hidden State. The actor maintains (h, c) between steps (*stateful*) and exposes a reset per episode, guaranteeing independence between trajectories.

Why this form. The sequence "LSTM $\rightarrow$ layer normalization $\rightarrow$ incremental head $\rightarrow$ final clipping $\rightarrow$ gate" separates responsibilities: the RNN summarizes history; the head decides on variations; the clipping ensures the contract; the gate introduces differentiable physics. The result is a policy that, by construction, cannot output actions out of range or propose fuel in invalid domains, and that retains a soft sensitivity to context changes.

9.2 Critics RNN

Purpose. To estimate $Q(z_t, a_t)$ under partial observability with an *independent* temporal representation, not conditioned by the actor’s memory.

Architecture

Normalization and Temporal Extraction. The observation is normalized with the same parameters as the actor and processed with an LSTM and *LayerNorm*, obtaining h_t .

Action-State Fusion. h_t is concatenated with the *normalized* action $a_t \in [-1, 1]^3$ and passed through an MLP head that produces a scalar Q . This fusion after the RNN allows the critic to model temporal observation patterns and, based on that, evaluate the marginal effect of the action.

Separate and Double Critic Memory. The actor and critics maintain independent memories; in addition, two critics with different parameters are implemented to reduce systematic value overestimation by using the minimum of both in the temporal targets. Although they share a philosophy, the critics do not share states or weights with the actor: they avoid representation bottlenecks and unwanted couplings.

9.3 Focused Comparison: from Initial to Current Architecture (actor)

For clarity, the changes that truly correct the problems detected in the *model-based* phase are highlighted:

Before (problematic).

- Scaling to $[-1, 1]$ was inconsistent with the environment (distribution mismatches).

- Final *linear* head that generated increments and was *summed* to the normalized input, leaving actions outside of $[-1, 1]$ (without final clipping).
- Physical rules applied as hard post-processing, without a differentiable gate in the policy space.

Now (corrected and suitable for RL).

- Coherent scaling with the environment: same constants, same range, same columns.
- Incremental head with tanh and a *final* tanh after the residual sum: the action is always in $[-1, 1]^3$ by construction.
- Soft gate for *mf* based on normalized *ice_{sp}*: respects physics without breaking the gradient; the environment will reinforce with its hard validation in real units.

10 Optimizing the Training Process: Efficiency and Stability

While the system’s components (environment, buffer, and network architectures) have been designed to be modular and functional, their direct application to full trajectories reveals two significant bottlenecks that compromise training viability:

- **Learning Complexity in Long Trajectories:** Our episodes consist of up to 1200 steps. Training with such long sequences introduces a high correlation in the data of each *batch*, which hinders the convergence and stability of learning.
- **Slow Data Collection:** The environment simulator (based on LSTMs for ICE and PG) is computationally intensive, becoming the main limiting factor in the speed at which the replay buffer can be filled.

To address these challenges, two strategic improvements are introduced: a redefinition of memory management in the buffer and a distributed architecture for experience collection.

10.1 Memory Management: The Overlapping Windows Buffer

Recurrent networks cannot learn from isolated transitions, but training with full trajectories is inefficient. The solution adopted is a compromise that preserves temporal context while improving data diversity: the buffer will store short, overlapping windows of a fixed length L .

The training process with these windows is structured in two phases:

- ***Burn-in* (B):** When sampling a window, the first B steps are executed in *forward* mode without gradient calculation. The purpose is to “warm up” the hidden states (h, c) of all recurrent networks (actor, critics, and their respective *target* networks), allowing the agent to reconstruct a realistic temporal context before starting learning.
- ***Unroll* (U):** The next U steps ($L = B + U$) are used to update the weights. During this phase, gradients are calculated and backpropagated through time, using an initial state already contextualized by the *burn-in* phase.

This approach maintains the sequential integrity necessary for RNNs, but by sampling short windows from different episodes, a much greater decorrelation in the *mini-batches* is achieved, which stabilizes and accelerates learning.

Handling Short Episodes: *Skip-tail*. An episode may end before reaching the maximum length (e.g., if the vehicle stabilizes). This would generate incomplete final windows (shorter than L). Since GPU training requires all tensors in a *batch* to have identical dimensions, these residual windows would cause errors. The *skip-tail* strategy solves this at the root: these incomplete final windows are systematically discarded, ensuring that the buffer only contains sequences of length L and guaranteeing compatibility with parallelized training.

10.2 Accelerating Experience: Actor-Learner Architecture

To solve the data collection bottleneck, a distributed Actor-Learner architecture is implemented. Responsibilities are separated into two distinct roles that run in parallel:

- **Learner (1, on GPU):** It is the brain of the system. Its only task is to continuously sample windows from the buffer and execute the training loop (calculating targets, backpropagating, and updating weights). By operating on a GPU, the efficiency of linear algebra is maximized.
- **Actors (multiple, on CPU):** They are the experience collectors. Multiple instances of the environment are run in parallel on the CPU cores. Each actor uses the current policy to interact with its copy of the environment, generate trajectories, and send the resulting windows to the centralized buffer.

This dissociation is key: while the *Learner* is busy training with the available data, the *Actors* are already generating the next wave of experience. This approach has drastically reduced cycle times, from approximately 1.5 hours to just 20 minutes per effective learning cycle.

Learning Cycle and Synchronization. The training cycle does not begin until the buffer has collected a minimum initial number of N windows, ensuring that the Learner has a sufficiently diverse experience base from the start. Once the learning phase begins, the Learner executes the refined TD3 loop: it samples windows, performs the burn-in, calculates the targets with target policy smoothing, updates the critics, and, with a delay, updates the actor and target networks (Polyak averaging). Periodically, the Actors pause their collection to receive the updated weights of the Learner’s policy, ensuring that exploration is carried out with an increasingly competent strategy.

11 Network Update: Actor and Critic Loss Functions

The learning mechanism of neural networks is governed by the minimization of a loss function. Although our approach is recurrent, the mathematical formulations for the actor and critic losses are conceptually identical to those of the original TD3 algorithm. The fundamental difference does not lie in the equations, but in the preparation of the input data: instead of operating on isolated transitions (o_t, a_t, r_t, o_{t+1}) , our loss functions process windows of sequences that have been previously contextualized through a *burn-in* phase.

11.1 Critic Loss

The objective of the critics is to learn the value function $Q(z_t, a_t)$, which estimates the expected return (sum of discounted future rewards) from executing action a_t from a belief state z_t . To do this, a temporal difference (TD) method is used.

Target Calculation: A target value y is constructed, which represents an improved estimate of the value Q . This is calculated using the *target* networks, which are more stable copies of the main networks.

$$y = r_t + \gamma \cdot Q'_{\min}(z'_{t+1}, \tilde{a}'_{t+1})$$

Where:

- r_t is the reward received at time t .
- γ is the discount factor.
- z'_{t+1} is the next belief state, obtained from the recurrent *target* network.
- $\tilde{a}'_{t+1}$ is the next action, proposed by the *target* actor and to which a small noise is added to smooth the value function (*Target Policy Smoothing*).
- $Q'_{\min}$ is the minimum value of the predictions of the two *target* critics, a key TD3 technique to combat value overestimation (*Clipped Double Q-Learning*).

Loss Function: The critic loss is the Mean Squared Error (MSE) between its current prediction and the target value y . Each critic is updated independently.

$$\mathcal{L}_{Q_i} = \mathbb{E}[(Q_i(z_t, a_t) - y)^2]$$

11.2 Actor Loss

The objective of the actor is to learn a deterministic policy $\pi(z_t)$ that maximizes the Q value estimated by the critics. That is, the actor seeks to generate actions that, according to the critic, lead to the best long-term results.

Loss Function: Instead of maximizing directly, the negative of the Q value is minimized. The actor’s loss is simply the average Q value of the actions that the policy proposes, evaluated by one of the critics (generally the first one).

$$\mathcal{L}_{\pi} = -\mathbb{E}[Q_1(z_t, \pi(z_t))]$$

By minimizing this expression, the optimizer adjusts the actor’s weights through gradient ascent, pushing the policy to generate actions with increasingly higher Q values. As is characteristic of TD3, this update is performed less frequently than that of the critics to ensure that the policy learns on a more stable value estimate.

12 Summary of System Architecture and Modularity

The developed system has been built following a principle of modularity, allowing its main components to be reused or adapted with relative ease. The role and degree of specificity of each key element are summarized below.

Environment (PG+ICE)

The environment, which encapsulates the hybrid vehicle simulation, has been designed as a ”black box” that complies with a standard Reinforcement Learning interface (with `step` and `reset` functions). Its internal logic is complex, but its external communication is generic. This makes it highly versatile, allowing any RL agent, regardless of its internal architecture (recurrent or not, *on-policy* or *off-policy*), to interact with it without the need for adaptations.

Replay Buffer

The buffer has been implemented to manage windows of experience instead of individual transitions, a necessity imposed by recurrent architectures. However, its design is not tied to our specific problem. It is reusable for any other RL project that employs a recurrent architecture (such as RDPG, R-PPO, etc.) and, therefore, requires sequential memory handling for training.

Actor and Critic Architectures

This is where we find different degrees of specificity:

Actor: The actor network is highly specific to the PG+ICE environment. Its architecture (inputs, outputs, and the differentiable physical gate) has been custom-designed to control the specific variables (`mf`, `brk`, `ice_sp`) and respect the physical constraints of our simulator.

Critic: In contrast, the critic network is more generic. Its only constraint is that it must be recurrent to be able to operate with the sequential output of the buffer. However, it does not contain any environment-specific logic, so it could be adapted to evaluate (state, action) pairs in other partially observable problems.

Complete Agent (Recurrent TD3)

The complete agent, which integrates the actor, critics, buffer, and the TD3 update algorithm, is specific to solving this problem. Its configuration (hyperparameters, update frequency, *burn-in* handling) has been optimized for the dynamics of the PG+ICE environment and the designed network architectures.

Evaluation Method

The evaluation system, largely inherited from the *model-based* phase, is mostly independent of the other components. The metrics used are standard control metrics that could be applied to any agent that attempts to solve the same task. The only specific part is the visualization of the final trajectories, since the generated graphs represent the physical variables (`v_ref`, `v`, `mf`, etc.) that are specific and exclusive to our environment.

13 Discussion of Results and Limitations

Despite the soundness of the theoretical framework and the implemented architecture, optimal results demonstrating a convergent and effective control policy were not obtained in the experimental phase. The agent, although it interacts with the environment, is unable to learn a coherent strategy to stabilize the vehicle’s speed at the reference setpoint.

The strategic reasoning behind the project—treating the problem as a POMDP, using a recurrent agent, optimizing training with a window buffer, and parallelizing data collection—remains theoretically sound and aligns with the most advanced practices in the field of Reinforcement Learning. Therefore, two main hypotheses are postulated to explain this discrepancy between the strategy and the results obtained:

1. **Computational Limitation and Sampling Efficiency:** Although the Actor-Learner architecture significantly accelerated the process, data collection remains the main bottleneck. Deep RL algorithms, especially in continuous control problems with complex dynamics, often require tens of millions of interactions with the environment to converge, **and we can only collect 300 samples in 1h**. It is very likely that, despite parallelization, the ability to collect the vast amount of diverse experience needed was below the threshold required for effective learning in a reasonable time. The *Learner* could be processing the same data over and over again before the *Actors* can fill the buffer with enough new information.
2. **Possibility of a Subtle Implementation Error:** RL systems are notoriously complex and sensitive to small coding errors that do not always generate obvious failures, but rather silently degrade learning. A latent error in the *burn-in* logic, in the correct propagation of hidden states, in the synchronization of the *target* networks, or even in the normalization of rewards, could be corrupting the learning signal received by the networks. Despite reviews, the existence of such an error that prevents the agent from making correct associations between its actions and long-term results cannot be completely ruled out.

In conclusion, the project establishes a correct and advanced work *pipeline*, but faces the practical barrier of computational resources or a possible implementation failure that could not be isolated.

References

A transition_function_model class

```
1 import numpy as np
2 import pandas as pd
3 import tensorflow as tf
4 from tensorflow.keras.models import clone_model, load_model
5 import joblib
6 import os
7
8 def get_model(model_path):
9     """
10     Loads a Keras model from the given path (must contain 'updated_model.keras')
11     and prints relevant information about the model.
12
13     Parameters:
14     model_path (str): Path to the directory containing the file 'updated_model.keras'
15     """
16     full_path = os.path.join(model_path, "model.h5")
17     full_path = os.path.normpath(full_path)
18     try:
19         model = load_model(full_path)
20     except Exception as e:
21         print(f"Error loading model from '{full_path}': {e}")
22     return model
23
24 def setup_transition_function_model(ICE_path, PG_path):
25     """
26     Configures the transition model by loading the required models and scalers.
27
28     Parameters:
29     ICE_path (str): Path to the ICE model.
30     PG_path (str): Path to the PG model.
31     """
32     ICE_model = get_model(ICE_path)
33     PG_model = get_model(PG_path)
34     ICE_scale_input = get_scaler(ICE_path, tipus="input")
35     ICE_scale_output = get_scaler(ICE_path, tipus="output")
36     PG_scale_input = get_scaler(PG_path, tipus="input")
37     PG_scale_output = get_scaler(PG_path, tipus="output")
38     return transition_function_model(
39         ICE_model, PG_model,
40         ICE_scale_input, ICE_scale_output,
41         PG_scale_input, PG_scale_output
42     )
43
44 def get_scaler(model_path, tipus="input"):
45     """
46     Loads a scaler (input or output) from a given path.
47
48     Parameters:
49     model_path (str): Path where the model is located.
50     tipus (str): 'input' or 'output', indicating which scaler to load.
51     """
52     if tipus == "input":
53         file = os.path.join(model_path, "input_scaler.lib")
54     elif tipus == "output":
55         file = os.path.join(model_path, "output_scaler.lib")
56     else:
57         raise ValueError("The 'tipus' parameter must be 'input' or 'output'")
58     return joblib.load(file)
59
60 class transition_function_model:
61     def __init__(self,
62                 ICE_model, Outlook_model,
63                 ICE_input_scaler, ICE_output_scaler,
64                 OUTLOOK_input_scaler, OUTLOOK_output_scaler,
65                 Torque=-22.33, NO_ini=0, CO_ini=0,
66                 velocity_ini=0, SOC_ini=0.7,
67                 show_progress=False):
68
69         # Frozen models
```

```

70     self.ICE_model      = clone_model(ICE_model)
71     self.ICE_model.set_weights(ICE_model.get_weights())
72     self.ICE_model.trainable = False
73
74     self.Outlook_model = clone_model(Outlook_model)
75     self.Outlook_model.set_weights(Outlook_model.get_weights())
76     self.Outlook_model.trainable = False
77
78     # Original scalers
79     self.ICE_input_scaler      = ICE_input_scaler
80     self.ICE_output_scaler     = ICE_output_scaler
81     self.OUTLOOK_input_scaler  = OUTLOOK_input_scaler
82     self.OUTLOOK_output_scaler = OUTLOOK_output_scaler
83
84     # MinMaxScaler parameters as TF tensors
85     self._ICE_in_scale      = tf.constant(ICE_input_scaler.scale_, dtype=tf.float32)
86     self._ICE_in_min       = tf.constant(ICE_input_scaler.min_,   dtype=tf.float32)
87     self._ICE_out_scale     = tf.constant(ICE_output_scaler.scale_, dtype=tf.float32)
88     self._ICE_out_min      = tf.constant(ICE_output_scaler.min_,  dtype=tf.float32)
89
90     self._PG_in_scale      = tf.constant(OUTLOOK_input_scaler.scale_, dtype=tf.
float32)
91     self._PG_in_min       = tf.constant(OUTLOOK_input_scaler.min_,   dtype=tf.
float32)
92     self._PG_out_scale    = tf.constant(OUTLOOK_output_scaler.scale_, dtype=tf.
float32)
93     self._PG_out_min      = tf.constant(OUTLOOK_output_scaler.min_,  dtype=tf.
float32)
94
95     # Initial auxiliary states
96     self.initial_ice_aux   = ICE_output_scaler.transform([[Torque, NO_ini, CO_ini
]]).reshape((1,1,3))
97     self.initial_outlook_aux = OUTLOOK_output_scaler.transform([[velocity_ini,
SOC_ini]]).reshape((1,1,2))
98     self.ice_aux          = tf.constant(self.initial_ice_aux, dtype=tf.float32)
99     self.outlook_aux      = tf.constant(self.initial_outlook_aux, dtype=tf.float32)
100
101     self.show_progress = show_progress
102     self.results = []
103
104     @staticmethod
105     def _scale_minmax(x, scale, min_):
106         return x * scale + min_
107
108     @staticmethod
109     def _descale_minmax(x_scaled, scale, min_):
110         return (x_scaled - min_) / scale
111
112     def reset_models(self):
113         if hasattr(self.ICE_model, 'reset_states'):
114             self.ICE_model.reset_states()
115         if hasattr(self.Outlook_model, 'reset_states'):
116             self.Outlook_model.reset_states()
117         self.ice_aux      = tf.constant(self.initial_ice_aux, dtype=tf.float32)
118         self.outlook_aux  = tf.constant(self.initial_outlook_aux, dtype=tf.float32)
119         self.results = []
120
121     def predict_ice(self, Speed_rpm, m_fuel_mg, T_amb_K, p_amb_bar, VTG):
122         x = tf.stack([Speed_rpm, m_fuel_mg, T_amb_K, p_amb_bar, VTG], axis=0)
123         x = tf.cast(x, tf.float32)
124         x_scaled = self._scale_minmax(x, self._ICE_in_scale, self._ICE_in_min)
125         x_scaled = tf.reshape(x_scaled, (1,1,5))
126         y_scaled = self.ICE_model([x_scaled, self.ice_aux], training=False)
127         y_scaled = tf.reshape(y_scaled, (3,))
128         y = self._descale_minmax(y_scaled, self._ICE_out_scale, self._ICE_out_min)
129         return tf.unstack(y, num=3)
130
131     def predict_PG(self, ICE_Speed_soll_rpm, EM2_Torque_Nm, ICE_Torque_Nm, Brake_perc)
:
132         x = tf.stack([ICE_Speed_soll_rpm, EM2_Torque_Nm, ICE_Torque_Nm, Brake_perc],
axis=0)

```

```

133     x = tf.cast(x, tf.float32)
134     x_scaled = self._scale_minmax(x, self._PG_in_scale, self._PG_in_min)
135     x_scaled = tf.reshape(x_scaled, (1,1,4))
136     y_scaled = self.Outlook_model([x_scaled, self.outlook_aux], training=False)
137     y_scaled = tf.reshape(y_scaled, (2,))
138     y = self._descale_minmax(y_scaled, self._PG_out_scale, self._PG_out_min)
139     return tf.unstack(y, num=2)
140
141     def predict(self,
142                 Speed_rpm, m_fuel_mg, T_amb_K, p_amb_bar, VTG,
143                 ICE_Speed_soll_rpm, Brake_perc, EM2_Torque_Nm):
144         ICE_Torque_Nm, NO_out, CO_out = self.predict_ice(
145             Speed_rpm, m_fuel_mg, T_amb_K, p_amb_bar, VTG)
146         Car_Vel_kmph, SOC = self.predict_PG(
147             ICE_Speed_soll_rpm, EM2_Torque_Nm, ICE_Torque_Nm, Brake_perc)
148         self.results.append([ICE_Torque_Nm, NO_out, CO_out, Car_Vel_kmph, SOC])
149         return ICE_Torque_Nm, NO_out, CO_out, Car_Vel_kmph, SOC

```

Listing 7: Python code for model integration and transition

B Experimentation Script with test

```

1  """
2  modular_transition.py
3  -----
4  Re-implementation of the evaluation flow from Modular.py, but using the
5  composite `transition_function_model` instead of loading two separate networks
6  from disk.
7
8  Output: generates `data_with_predictions_output2.csv` along with metrics and
9  plots in a subdirectory `plots/results_<timestamp>`, just like the original script.
10
11  Prerequisites
12  ~~~~~
13  1. Place `transition_function_model.py` in the same directory as this script
14     (or in the PYTHONPATH).
15  2. Have the directories with the pre-trained models:
16     ICE_Model_M1
17     PG_Model_M1.1_without_EM1_Torque
18     (passed below as `ICE_MODEL_PATH` and `PG_MODEL_PATH`).
19  3. Keep `config.txt`. The keys `inputs`, `outputs`,
20     `initial_outputs`, etc., are still read, but `directory` is ignored.
21  4. Install dependencies: tensorflow, scikit-learn, pandas, numpy,
22     matplotlib, plotly, graphviz (same as `Modular.py`).
23
24  Key differences from Modular.py
25  ~~~~~
26  * There is only one model `t_function` with the `.predict(...)` method,
27    which simultaneously returns the ICE block outputs (Torque, NO, CO) and the
28    PG block outputs (Car_Speed, SOC).
29  * Internally we clone and reset states with `t_function.reset_models()` once
30    per simulation (before the timestep loop).
31  * In the output CSV we keep the same column format as before, with the
32    `_pred` suffix, so that downstream pipelines continue to work unchanged.
33  """
34
35  import os
36  import time
37  import shutil
38  from datetime import datetime
39
40  import numpy as np
41  import pandas as pd
42  import matplotlib.pyplot as plt
43  import plotly.graph_objs as go
44
45  import warnings
46  from sklearn.exceptions import DataConversionWarning
47
48  from transition_function_model_2 import (

```

```

49     setup_transition_function_model,
50 )
51
52 #####
53 # 1. Parsers and utilities (identical to Modular.py, except minor adjustments) #
54 #####
55
56 CONFIG_KEYS = {"inputs", "outputs", "type", "initial_outputs", "manual_initialization"
57               }
58
59 def parse_config(config_path: str):
60     """Reads the configuration file but **ignores** the `directory` key."""
61     cfg: dict[str, dict] = {}
62     current = None
63     with open(config_path, "r", encoding="utf-8") as fh:
64         for raw in fh:
65             line = raw.strip()
66             if not line:
67                 continue
68             if not any(line.startswith(k) for k in CONFIG_KEYS):
69                 current = line
70                 cfg[current] = {}
71                 continue
72             key, value = map(str.strip, line.split("=", 1))
73             if key == "inputs":
74                 cfg[current]["inputs"] = [x.strip() for x in value.split(",")]
75             elif key == "outputs":
76                 cfg[current]["outputs"] = [x.strip() for x in value.split(",")]
77             elif key == "type":
78                 cfg[current]["type"] = value
79             elif key == "initial_outputs":
80                 cfg[current]["initial_outputs"] = [float(x.strip()) for x in value.
81                 split(",")]
82             elif key == "manual_initialization":
83                 manual_init = {}
84                 for pair in value.split(","):
85                     idx, val = map(str.strip, pair.split(":"))
86                     manual_init[int(idx)] = float(val)
87                 cfg[current]["manual_initialization"] = manual_init
88     return cfg
89
90 def calculate_mae(pred, true):
91     """Compute Mean Absolute Error, ignoring non-finite values."""
92     mask = np.isfinite(pred) & np.isfinite(true)
93     return np.mean(np.abs(pred[mask] - true[mask]))
94
95 def calculate_mape(pred, true):
96     """Compute Mean Absolute Percentage Error, ignoring zeros and non-finite."""
97     mask = np.isfinite(pred) & np.isfinite(true) & (true != 0)
98     return np.mean(np.abs((true[mask] - pred[mask]) / true[mask])) * 100
99
100 def calculate_max_ae(pred, true):
101     """Compute Maximum Absolute Error, ignoring non-finite values."""
102     mask = np.isfinite(pred) & np.isfinite(true)
103     return np.max(np.abs(pred[mask] - true[mask]))
104
105 def calculate_max_re(pred, true):
106     """Compute Maximum Relative Error (%), ignoring zeros and non-finite."""
107     mask = np.isfinite(pred) & np.isfinite(true) & (true != 0)
108     return np.max(np.abs((true[mask] - pred[mask]) / true[mask])) * 100
109
110 #####
111 # 2. Dataset loading
112 #####
113
114 def load_dataset(csv_path: str):
115     """Load the CSV and clean headers (same as Modular.py)."""
116     df = pd.read_csv(csv_path, delimiter=",", encoding="latin1")
117     # Map from "clean" name to original (for columns with units)
118     clean_cols = {c.split("(")[0].strip(): c for c in df.columns}
119     return df, clean_cols

```

```

118
119 #####
120 # 3. Inference loop using transition_function_model #
121 #####
122
123 ICE_MODEL_PATH = "XXX"
124 PG_MODEL_PATH = "XXX"
125
126 def run_simulation(config: dict, df: pd.DataFrame, clean_cols: dict[str, str]):
127     """Runs the simulation and returns predictions in a dict."""
128     # 3.1 Initialize composite model -----
129     t_function = setup_transition_function_model(ICE_MODEL_PATH, PG_MODEL_PATH)
130     # Reset states in case of multiple calls
131     t_function.reset_models()
132
133     # 3.2 Pre-allocate arrays -----
134     n = len(df)
135     preds = {
136         "ICE_Torque": np.empty((n, 1)),
137         "NO_out_m_pred": np.empty((n, 1)),
138         "NO2_out_m_pred": np.empty((n, 1)),
139         "CO_out_m_pred": np.empty((n, 1)),
140         "CO2_out_m_pred": np.empty((n, 1)),
141         "Car_Speed": np.empty((n, 1)),
142         "SOC": np.empty((n, 1)),
143     }
144
145     # 3.3 Time-step iteration -----
146     tic_total = time.time()
147     for t in range(n):
148         row = df.iloc[t]
149         ice_speed_soll = row[clean_cols["ICE_Speed_soll"]]
150         fuel = row[clean_cols["fuel"]]
151         T_amb = row[clean_cols["T_amb"]]
152         p_amb = row[clean_cols["p_amb"]]
153         brake = row[clean_cols["Brake"]]
154         em2_torque = row[clean_cols["EM2_Torque"]]
155
156         # ---- Single aggregated prediction -----
157         # Unpacks: Torque, NO, CO, Car_Speed, SOC
158         torque, NO_out, NO2_out, CO_out, CO2_out, car_speed, soc = t_function.predict(
159             Speed_rpm=ice_speed_soll,
160             m_fuel_mg=fuel,
161             T_amb_K=T_amb,
162             p_amb_bar=p_amb,
163             ICE_Speed_soll_rpm=ice_speed_soll,
164             Brake_perc=brake,
165             EM2_Torque_Nm=em2_torque
166         )
167
168         preds["ICE_Torque"][t] = torque
169         preds["NO_out_m_pred"][t] = NO_out
170         preds["NO2_out_m_pred"][t] = NO2_out
171         preds["CO_out_m_pred"][t] = CO_out
172         preds["CO2_out_m_pred"][t] = CO2_out
173         preds["Car_Speed"][t] = car_speed
174         preds["SOC"][t] = soc
175
176     toc_total = time.time() - tic_total
177     print(f"Total simulation time: {toc_total:.3f} s ( {toc_total/n:.6f} s/step)")
178     return preds
179
180 #####
181 # 4. Post-processing: metrics, plots, CSV #
182 #####
183
184 def add_predictions_to_df(df: pd.DataFrame, preds: dict, clean_cols: dict[str, str]):
185     """Append prediction columns to the DataFrame."""
186     for k, arr in preds.items():
187         df[f"{k}_pred"] = arr
188     return df

```



```

189
190 def evaluate_and_plot(df: pd.DataFrame, preds: dict, clean_cols: dict[str, str],
191                       output_dir: str):
192     """Compute evaluation metrics, generate plots, and save outputs."""
193     maes = {}
194     mapes = {}
195     max_aes = {}
196     max_res = {}
197
198     os.makedirs(output_dir, exist_ok=True)
199
200     for k, arr in preds.items():
201         true_vals = df[clean_cols.get(k, k)].values.reshape(-1, 1)
202
203         mae = calculate_mae(arr, true_vals)
204         mape = calculate_mape(arr, true_vals)
205         max_ae = calculate_max_ae(arr, true_vals)
206         max_re = calculate_max_re(arr, true_vals)
207
208         maes[k] = mae
209         mapes[k] = mape
210         max_aes[k] = max_ae
211         max_res[k] = max_re
212
213         # ----- Interactive trace plot -----
214         x = np.arange(len(arr))
215         fig = go.Figure()
216         fig.add_trace(go.Scatter(x=x, y=true_vals.flatten(), mode="lines", name="True"
217                                 ))
218         fig.add_trace(go.Scatter(x=x, y=arr.flatten(), mode="lines", name="Pred"))
219         fig.update_layout(title=k, xaxis_title="timestep", yaxis_title=k)
220         fig.write_html(os.path.join(output_dir, f"{k}_interactive.html"))
221
222         # ----- Static scatter plot -----
223         plt.figure(figsize=(6, 4))
224         plt.scatter(true_vals, arr, s=15)
225         lims = [min(true_vals.min(), arr.min()), max(true_vals.max(), arr.max())]
226         plt.plot(lims, lims, "k-")
227         plt.xlabel("True")
228         plt.ylabel("Pred")
229         plt.title(k)
230         plt.tight_layout()
231         plt.savefig(os.path.join(output_dir, f"{k}_scatter.png"))
232         plt.close()
233
234         print(f"{k}: MAE={mae:.4f} MAPE={mape:.2f}% MaxAE={max_ae:.4f} MaxRE={
235               max_res:.2f}%")
236
237     return maes, mapes, max_aes, max_res
238
239 def average_and_export(config_path: str, csv_path: str, plots_dir: str = "plots", runs
240                        : int = 5):
241     """Run multiple simulations, average results, and export CSV and plots."""
242     # 1) Read config and data
243     cfg = parse_config(config_path)
244     df, clean_cols = load_dataset(csv_path)
245
246     # 2) Run N simulations
247     all_preds = []
248     for i in range(runs):
249         print(f"Running simulation {i + 1} of {runs}...")
250         preds = run_simulation(cfg, df, clean_cols)
251         all_preds.append(preds)
252
253     # 3) Stack and average
254     avg_preds = {}
255     for k in all_preds[0]:
256         stacked = np.hstack([p[k] for p in all_preds])
257         avg_preds[k] = np.mean(stacked, axis=1, keepdims=True)
258
259     # 4) Save results

```

```

256 timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
257 out_dir = os.path.join(plots_dir, f"results_{timestamp}")
258 os.makedirs(out_dir, exist_ok=True)
259 shutil.copy(config_path, os.path.join(out_dir, os.path.basename(config_path)))
260
261 df = add_predictions_to_df(df, avg_preds, clean_cols)
262 df.to_csv(os.path.join(out_dir, "data_with_predictions_output_mikel.csv"), index=
False)
263 evaluate_and_plot(df, avg_preds, clean_cols, out_dir)
264 print(f"Average CSV saved to {os.path.join(out_dir, '
data_with_predictions_output.csv')}")
265
266 if __name__ == "__main__":
267     warnings.filterwarnings('ignore',
268         message='X does not have valid feature names, but MinMaxScaler was fitted with
feature names',
269         category=UserWarning
270     )
271     CONFIG_PATH = 'xxx.txt'
272     CSV_PATH = 'yyy.csv'
273     average_and_export(CONFIG_PATH, CSV_PATH, runs=1)

```

Listing 8: Python code for validation

C Experimentation Script with Case Studies

```

1 # Setup
2
3 from plotly.subplots import make_subplots
4 import plotly.graph_objects as go
5 import tensorflow as tf
6 import os
7 import pandas as pd
8 import plotly.express as px
9 from transition_function_model_v2 import (
10     setup_transition_function_model,
11 )
12
13
14 def set_experiments(
15     name: str,
16     ruta_ICE: str,
17     ruta_PG: str,
18     experiment_runner,          # function that runs the experiment
19     create_plot_fn,
20     output_root: str = "Model_Discussion"
21 ):
22     """
23     Runs an experiment and saves results + plot.
24
25     Parameters
26     -----
27     name           : Name of the subfolder (e.g. "Case1").
28     ruta_ICE       : Path to the ICE model (.h5 or folder).
29     ruta_PG       : Path to the PG model (.h5 or folder).
30     experiment_runner : Callable[[transition_function_model], pd.DataFrame]
31                     : Function that takes 'trans_func' and returns a DataFrame.
32     create_plot_fn  : Callable[[pd.DataFrame, str], plotly.graph_objects.Figure]
33                     : Function responsible for building the HTML plot.
34     output_root     : Root folder where results will be saved.
35     """
36
37     # 1      Output folder
38     folder = os.path.join(output_root, name)
39     os.makedirs(folder, exist_ok=True)
40
41     # 2      Transition model
42     trans_func = setup_transition_function_model(ruta_ICE, ruta_PG)
43
44     # 3      Run the experiment

```

```

45     df = experiment_runner(trans_func)
46
47     # 4      Save CSV
48     df.to_csv(os.path.join(folder, "raw_results.csv"), index=False)
49
50     # 5      Create and save the plot using the supplied function
51     fig = create_plot_fn(df, f"{name}      Results")
52
53     # Save as HTML
54     html_path = os.path.join(folder, f"traj_{name}.html")
55     fig.write_html(html_path)
56
57     #      Save as PNG
58     png_path = os.path.join(folder, f"traj_{name}.png")
59     fig.write_image(png_path)
60
61     print(f"      Results, HTML plot, and PNG saved to      {folder}  ")
62
63     return folder, df
64
65
66 # Experiment 1: NOx at idle
67 def plot_nox_only(df, title):
68     fig = px.line(
69         df, x="step", y="nox",
70         title=title,
71         labels={"step": "Simulation step", "nox": "NOx [g/s]"}
72     )
73     fig.update_layout(hovermode="x unified")
74     return fig
75
76 def experiment_1(trans_func, steps: int = 5000):
77     """
78     Case 1: Idling at 1000 rpm with 3 mg of fuel.
79     Captures and returns only the NOx emission over the steps.
80     """
81     data = {"step": [], "nox": []}
82     ice_sp_t = tf.constant(1000.0, dtype=tf.float32)
83     fuelmass_t = tf.constant(3.0, dtype=tf.float32)
84     T_amb_t = tf.constant(298.0, dtype=tf.float32)
85     p_amb_t = tf.constant(1.0, dtype=tf.float32)
86
87     for k in range(steps):
88         torque, nox, *_ = trans_func.predict_ice(
89             ice_sp_t, fuelmass_t, T_amb_t, p_amb_t
90         )
91         data["step"].append(k)
92         data["nox"].append(float(nox.numpy()))
93
94     return pd.DataFrame(data)
95
96 # Example call
97 folder1, df_case1 = set_experiments(
98     name="Case1",
99     ruta_ICE="../../models_markus/ICE_Model_Update_01",
100     ruta_PG="../../models_markus/PG_Model_M1.1_without_EM1_Torque",
101     experiment_runner=experiment_1,
102     create_plot_fn=plot_nox_only
103 )
104
105
106 # Experiment 2: Torque in load-step
107 def experiment_2(
108     trans_func,
109     total_steps: int = 2500,
110     interval: int = 200,
111     fuelmass_profile=[3, 10, 20, 30, 40, 50, 60, 70, 5],
112     ice_speed: float = 2500.0,
113     T_amb_K: float = 298.0,
114     p_amb_bar: float = 1.0
115 ):

```

```

116 data = {"step": [], "mf": [], "torque": []}
117 ice_sp_t = tf.constant(ice_speed, dtype=tf.float32)
118 T_amb_t = tf.constant(T_amb_K, dtype=tf.float32)
119 p_amb_t = tf.constant(p_amb_bar, dtype=tf.float32)
120
121 for step in range(total_steps):
122     idx = step // interval
123     mf_val = fuelmass_profile[idx] if idx < len(fuelmass_profile) else
fuelmass_profile[-1]
124     mf_t = tf.constant(mf_val, dtype=tf.float32)
125     torque, *_ = trans_func.predict_ice(ice_sp_t, mf_t, T_amb_t, p_amb_t)
126     data["step"].append(step)
127     data["mf"].append(mf_val)
128     data["torque"].append(float(torque.numpy()))
129
130 return pd.DataFrame(data)
131
132 def plot_torque_only(df, title="ICE Torque (load-step)":
133     fig = px.line(df, x="step", y="torque", title=title,
134                   labels={"step": "Time [steps]", "torque": "ICE Torque [Nm]})
135     fig.update_layout(hovermode="x unified")
136     return fig
137
138 # Example call
139 folder2, df_case2 = set_experiments(
140     name="Case2",
141     ruta_ICE="./models_markus/ICE_Model_Update_01",
142     ruta_PG="./models_markus/PG_Model_M1.1_without_EM1_Torque",
143     experiment_runner=experiment_2,
144     create_plot_fn=plot_torque_only
145 )
146
147
148 # Experiment 3: Combined SoC and speed
149 def experiment_3(
150     trans_func,
151     total_steps: int = 5000,
152     interval: int = 400,
153     em2_torque_profile=[50, 250, 400],
154     ice_speed: float = 0.0,
155     fuelmass: float = 0.0,
156     brake: float = 0.0,
157     T_amb_K: float = 298.0,
158     p_amb_bar: float = 1.0
159 ):
160     data = {"step": [], "em2_torque": [], "soc": [], "vel": []}
161     ice_sp_t = tf.constant(ice_speed, dtype=tf.float32)
162     fuel_t = tf.constant(fuelmass, dtype=tf.float32)
163     T_amb_t = tf.constant(T_amb_K, dtype=tf.float32)
164     p_amb_t = tf.constant(p_amb_bar, dtype=tf.float32)
165     brake_t = tf.constant(brake, dtype=tf.float32)
166     n_levels = len(em2_torque_profile)
167
168     for step in range(total_steps):
169         idx = min(step // interval, n_levels - 1)
170         em2_t = tf.constant(em2_torque_profile[idx], dtype=tf.float32)
171
172         torque_ice, *_ = trans_func.predict_ice(ice_sp_t, fuel_t, T_amb_t, p_amb_t)
173         vel_out, soc_pred = trans_func.predict_PG(ice_sp_t, em2_t, torque_ice, brake_t
174         )
175
176         data["step"].append(step)
177         data["em2_torque"].append(float(em2_torque_profile[idx]))
178         data["soc"].append(float(soc_pred.numpy()))
179         data["vel"].append(float(vel_out.numpy()))
180
181     return pd.DataFrame(data)
182
183 def plot_soc_vel(df, title="Case 3: SoC & Car Speed vs Time)":
184     fig = make_subplots(specs=[[{"secondary_y": True}]]
185     fig.add_trace(go.Scatter(x=df["step"], y=df["soc"], name="SoC_pred"), secondary_y=

```

```

False)
185 fig.add_trace(go.Scatter(x=df["step"], y=df["vel"], name="Car_Speed_pred"),
secondary_y=True)
186 fig.update_layout(title=title, xaxis_title="Time [steps]", hovermode="x unified")
187 fig.update_yaxes(title_text="SoC [fraction]", secondary_y=False)
188 fig.update_yaxes(title_text="Car Velocity [km/h]", secondary_y=True)
189 return fig
190
191 # Example call
192 folder3, df_case3 = set_experiments(
193     name="Case3_v2",
194     ruta_ICE=" ../models_markus/ICE_Model_Update_01",
195     ruta_PG=" ../models_markus/PG_v2",
196     experiment_runner=experiment_3,
197     create_plot_fn=plot_soc_vel
198 )

```

Listing 9: Python code for the `set_experiments` function and case studies

D Controller Architecture V1

```

1 def build_primer_model_stateful(units: int = 64*2) -> tf.keras.Model:
2     inp = tf.keras.Input(batch_shape=(1, 1, 6), name="ctrl_in")
3
4     # Stateful LSTM layer
5     x = tf.keras.layers.LSTM(units, stateful=True, return_sequences=False)(inp)
6
7     # Normalize the LSTM output
8     x = tf.keras.layers.LayerNormalization()(x)
9
10    # Intermediate dense layer
11    x = tf.keras.layers.Dense(units // 2, activation="tanh")(x)
12
13    # Output layer
14    out = tf.keras.layers.Dense(4, activation="linear", name="deltas")(x)
15
16    model = tf.keras.Model(inp, out, name="controller_stateful")
17    return model

```

Listing 10: Python implementation of the controller's stateful architecture

E Controller Training Script V1

```

1 import tensorflow as tf
2
3 def sample_init_state():
4     """
5     Returns a tf.Tensor of shape (5,) with random values in the ranges:
6     - mf:      [3, 70]
7     - vtg:     [0.23, 0.9]
8     - brk:     [0, 100]
9     - ice_sp:  [900, 4500]
10    """
11    mf      = tf.random.uniform([], minval=3.0,      maxval=70.0,      dtype=tf.float32)
12    vtg     = tf.random.uniform([], minval=0.23,     maxval=0.9,      dtype=tf.float32)
13    brk     = tf.random.uniform([], minval=0.0,      maxval=100.0,    dtype=tf.float32)
14    ice_sp  = tf.random.uniform([], minval=900.0,    maxval=4500.0,   dtype=tf.float32)
15
16    state = tf.stack([mf, vtg, brk, ice_sp], axis=0)
17    return state
18
19
20
21 def simulate_trajectory(controller, trans_func, init_state, clipping, K=1200,
22     vel_target=70.0):
23     # Reset ICE/PG and controller states
24     tf.py_function(trans_func.reset_models, [], [])

```

```

24 tf.py_function(controller.reset_states, [], [])
25
26 # Prepare storage for time series data
27 data = {k: [] for k in ("mf", "vtg", "brk", "ice_sp", "torque", "vel_out", "nox", "co")}
28
29 # Constant ambient conditions
30 p_amb_bar = tf.constant(1.0, dtype=tf.float32)
31 T_amb_K = tf.constant(298.0, dtype=tf.float32)
32
33 # Unpack initial state
34 _, mf, vtg, brk, ice_sp = tf.unstack(init_state)
35
36 # First ICE prediction
37 torque, _, _ = trans_func.predict_ice(ice_sp, mf, T_amb_K, p_amb_bar, vtg)
38 if clipping:
39     torque = tf.clip_by_value(torque, -50.0, 300.0)
40
41 # First PG prediction
42 vel, _ = trans_func.predict_PG(ice_sp, 0.0, torque, brk)
43
44 for _ in range(K):
45     # Build controller input
46     x = tf.stack([vel_target, vel, mf, vtg, brk, ice_sp])
47     inp = tf.reshape(x, (1,1,6)) # shape (1,1,6)
48
49     # Controller forward pass (no training)
50     out = controller(inp, training=False)[0] # shape (4,)
51     mf, vtg, brk, ice_sp = tf.unstack(out, num=4)
52
53     # ICE prediction
54     torque, nox, co = trans_func.predict_ice(ice_sp, mf, T_amb_K, p_amb_bar, vtg)
55     if clipping:
56         torque = tf.clip_by_value(torque, -50.0, 300.0)
57
58     # Prevent operation below idle
59     mask_ice = ice_sp < 900.0
60     torque = tf.where(mask_ice, tf.zeros_like(torque), torque)
61     mf = tf.where(mask_ice, tf.zeros_like(mf), mf)
62
63     # PG prediction
64     vel_out, _ = trans_func.predict_PG(ice_sp, 0.0, torque, brk)
65     vel = vel_out
66
67     # Record data
68     data["torque"].append(torque.numpy())
69     data["mf"].append(mf.numpy())
70     data["vtg"].append(vtg.numpy())
71     data["brk"].append(brk.numpy())
72     data["ice_sp"].append(ice_sp.numpy())
73     data["vel_out"].append(vel_out.numpy())
74     data["nox"].append(nox.numpy())
75     data["co"].append(co.numpy())
76
77 df = pd.DataFrame(data)
78 df["step"] = df.index
79 return df
80
81
82 def create_interactive_plot(df, title):
83     fig = px.line(df, x="step",
84                   y=["mf", "vtg", "brk", "ice_sp", "torque", "vel_out", "nox", "co"],
85                   title=title)
86     # Hide output traces by default
87     for tr in fig.data:
88         if tr.name in ("vel_out", "nox", "co"):
89             tr.visible = "legendonly"
90     buttons = [
91         dict(label="States",
92              method="update",
93              args=[{"visible": [t.name in ("mf", "vtg", "brk", "ice_sp", "torque") for t
in fig.data]}],

```

```

94         {"title": f"{title}      States"}]],
95     dict(label="Outputs",
96         method="update",
97         args=[{"visible": [t.name in ("vel_out", "nox", "co") for t in fig.data]},
98             {"title": f"{title}      Outputs"}]],
99     ]
100     fig.update_layout(updatemenus=[dict(type="buttons", buttons=buttons, direction="
101     right")])
102     return fig
103
104 #####
105 # eval_loss_components      optimized version (      1015      faster on GPU)
106 #####
107
108 @tf.function()
109 def eval_loss_components(controller,
110     trans_func,
111     N=20,
112     total_steps=1200,
113     last_steps=200,
114     clipping=True,
115     vel_target=75.0,
116     warmup_steps=200):
117     """
118     Runs N trajectories and returns the average of:
119     1) |vel_target      vel_out|
120     2) NOx
121     3) CO
122
123     Uses the same simulation loop as rollout_and_loss, but without training,
124     and accumulates each loss component.
125     """
126     # Constants
127     p_amb_bar = tf.constant(1.0, tf.float32)
128     T_amb_K   = tf.constant(298.0, tf.float32)
129     threshold = total_steps - last_steps
130
131     # Global accumulators
132     g_vel, g_nox, g_co = 0.0, 0.0, 0.0
133
134     for traj in range(N):
135         # Reset models and states
136         tf.py_function(trans_func.reset_models, [], [])
137         tf.py_function(controller.reset_states, [], [])
138
139         # Random initial state
140         init_state = sample_init_state()
141         _, mf, vtg, brk, ice_sp = tf.unstack(init_state)
142
143         # First prediction
144         torque, _, _ = trans_func.predict_ice(ice_sp, mf, T_amb_K, p_amb_bar, vtg)
145         if clipping:
146             torque = tf.clip_by_value(torque, -50.0, 300.0)
147         vel, _ = trans_func.predict_PG(ice_sp, 0.0, torque, brk)
148
149         # Per-trajectory accumulators
150         t_vel, t_nox, t_co = 0.0, 0.0, 0.0
151
152         for i in range(total_steps):
153             # Controller input
154             x = tf.stack([vel_target, vel, mf, vtg, brk, ice_sp])
155             inp = tf.reshape(x, (1,1,6))
156
157             # Forward pass
158             out = controller(inp, training=False)[0]
159             mf, vtg, brk, ice_sp = tf.unstack(out, num=4)
160
161             # ICE + PG prediction
162             torque, nox, co = trans_func.predict_ice(ice_sp, mf, T_amb_K, p_amb_bar,
vtg)

```

```

163         if clipping:
164             torque = tf.clip_by_value(torque, -50.0, 300.0)
165             vel_out, _ = trans_func.predict_PG(ice_sp, 0.0, torque, brk)
166
167             # Warmup mask
168             w = tf.minimum(1.0, tf.cast(i, tf.float32) / tf.cast(warmup_steps, tf.
float32))
169
170             # Accumulate only in the last `last_steps`
171             t_vel += w * tf.abs(vel_target - vel_out)
172             t_nox += w * nox
173             t_co += w * co
174
175             vel = vel_out
176
177             inv_last = 1.0 / tf.cast(last_steps, tf.float32)
178             g_vel += t_vel * inv_last
179             g_nox += t_nox * inv_last
180             g_co += t_co * inv_last
181
182             inv_N = 1.0 / tf.cast(N, tf.float32)
183             return g_vel * inv_N, g_nox * inv_N, g_co * inv_N

```

Listing 11: Controller Training v1

F Controller Architecture V2

```

1
2 import os, joblib, copy
3 import numpy as np
4 from sklearn.preprocessing import MinMaxScaler
5
6 def get_scaler(model_path, tipus="input"):
7     """
8     Loads a scaler (input or output) from a given path and displays relevant
9     information.
10
11     Parameters:
12     model_path (str): Path where the model is located.
13     tipus (str): 'input' or 'output' indicating which scaler to load.
14
15     Returns:
16     The loaded scaler object.
17     """
18     if tipus == "input":
19         file = os.path.join(model_path, "input_scaler.lib")
20     elif tipus == "output":
21         file = os.path.join(model_path, "output_scaler.lib")
22     else:
23         raise ValueError("The 'tipus' parameter must be 'input' or 'output'")
24
25     scaler = joblib.load(file)
26     return scaler
27
28 class ScalerLibBuilder:
29     """
30     Builds 'collage' scalers by combining columns from other MinMaxScalers.
31     Always saves the .lib files in ./escalados.
32     """
33
34     def __init__(self, ruta_ICE: str, ruta_PG: str):
35         # Load original scalers
36         self.ice_in = get_scaler(ruta_ICE, "input")
37         self.ice_out = get_scaler(ruta_ICE, "output")
38         self.pg_in = get_scaler(ruta_PG, "input")
39         self.pg_out = get_scaler(ruta_PG, "output")
40
41         # Map of available scaler libraries
42         self._av = {

```



```

43         "ICE_input": self.ice_in,
44         "ICE_output": self.ice_out,
45         "PG_input": self.pg_in,
46         "PG_output": self.pg_out
47     }
48
49     # Prepare output folder
50     self.save_folder = "escalados"
51     os.makedirs(self.save_folder, exist_ok=True)
52
53     def _slice_attr(self, attr_arr, idx):
54         """
55         Returns attr_arr[idx] if it exists, otherwise None
56         (e.g. for feature_names_in_).
57         """
58         return attr_arr[idx] if attr_arr is not None else None
59
60     def create_lib(self, new_lib_name: str,
61                   selections: list[tuple[str, int]]):
62         """
63         Constructs and saves a new MinMaxScaler in the 'escalados' folder,
64         copying all internal attributes from the original scaler(s) for each
65         selected variable.
66
67         Available variables in each original library (1-based indices):
68
69         ICE_input:
70             1 = Speed_rpm
71             2 = m_fuel_mg
72             3 = T_amb_K
73             4 = p_amb_bar
74             5 = Rack_Pos_1
75
76         ICE_output:
77             1 = Torque_Nm
78             2 = NO_out_1
79             3 = CO_out_1
80
81         PG_input:
82             1 = ICE_Speed_soll_rpm
83             2 = EM2_Torque_Nm
84             3 = ICE_Torque_Nm
85             4 = Brake_perc
86
87         PG_output:
88             1 = Car_Speed_kmph
89             2 = SOC_1
90
91         Args:
92             new_lib_name: Base name (without extension) of the resulting .lib file.
93             selections: List of tuples (key, position_1based), where
94                 - key is one of "ICE_input"/"ICE_output"/"PG_input"/
95                 PG_output"
96                 - position_1based is the 1-based index of the variable.
97
98         Returns:
99             The complete MinMaxScaler object, with all internal attributes
100             cloned and sliced according to the selection.
101         """
102         # Pre-allocate arrays for the new scaler
103         cols = len(selections)
104         scale = np.empty(cols)
105         min_ = np.empty(cols)
106         data_min = np.empty(cols)
107         data_max = np.empty(cols)
108         data_range = np.empty(cols)
109         feat_names = [] # may remain empty if names are not available
110
111         clip_val = None
112         f_range = None
113         n_samples = None

```

```

113
114     # Slice each selected column
115     for k, (key, pos1) in enumerate(selections):
116         src = self._av[key]
117         i = pos1 - 1 # convert to 0-based index
118         scale[k] = src.scale_[i]
119         min_[k] = src.min_[i]
120         data_min[k] = src.data_min_[i]
121         data_max[k] = src.data_max_[i]
122         data_range[k] = src.data_range_[i]
123
124     # Collect feature names if available
125     if hasattr(src, "feature_names_in_"):
126         feat_names.append(src.feature_names_in_[i])
127     else:
128         feat_names = None # drop names if any column lacks them
129
130     # Copy metadata once (shared across all columns)
131     clip_val = src.clip
132     f_range = src.feature_range
133     n_samples = src.n_samples_seen_
134
135     # Build the new scaler
136     new_scaler = MinMaxScaler(feature_range=f_range, clip=clip_val, copy=True)
137     # Manually assign internal attributes
138     new_scaler.scale_ = scale
139     new_scaler.min_ = min_
140     new_scaler.data_min_ = data_min
141     new_scaler.data_max_ = data_max
142     new_scaler.data_range_ = data_range
143     new_scaler.n_features_in_ = cols
144     new_scaler.n_samples_seen_ = n_samples
145
146     if feat_names is not None:
147         new_scaler.feature_names_in_ = np.array(feat_names, dtype=object)
148
149     # Save the new scaler
150     out_path = os.path.join(self.save_folder, f"{new_lib_name}.lib")
151     joblib.dump(new_scaler, out_path)
152     print(f"    '{new_lib_name}.lib' saved in '{self.save_folder}/'")
153
154     return new_scaler

```

Listing 12: Python implementation of the improved controller architecture

G Custom Scalers Collage

```

1 import os, joblib, copy
2 import numpy as np
3 from sklearn.preprocessing import MinMaxScaler
4
5 def get_scaler(model_path, tipus="input"):
6     """
7     Load a scaler (input or output) from a path and display relevant info.
8
9     Parameters:
10     model_path (str): Path where the model is located.
11     tipus (str): 'input' or 'output' indicating which scaler to load.
12
13     Returns:
14     The loaded scaler object.
15     """
16     if tipus == "input":
17         file = os.path.join(model_path, "input_scaler.lib")
18     elif tipus == "output":
19         file = os.path.join(model_path, "output_scaler.lib")
20     else:
21         raise ValueError("The 'tipus' parameter must be 'input' or 'output'")
22
23     scaler = joblib.load(file)

```

```

24     return scaler
25
26
27 class ScalerLibBuilder:
28     """
29     Builds a 'collage' of scalers by combining columns from existing MinMaxScalers.
30     Always saves the resulting .lib files under ./escalados.
31     """
32
33     def __init__(self, ruta_ICE: str, ruta_PG: str):
34         # Load the original scalers
35         self.ice_in = get_scaler(ruta_ICE, "input")
36         self.ice_out = get_scaler(ruta_ICE, "output")
37         self.pg_in = get_scaler(ruta_PG, "input")
38         self.pg_out = get_scaler(ruta_PG, "output")
39
40         # Map of available scaler libraries
41         self._av = {
42             "ICE_input": self.ice_in,
43             "ICE_output": self.ice_out,
44             "PG_input": self.pg_in,
45             "PG_output": self.pg_out
46         }
47
48         # Prepare output directory
49         self.save_folder = "escalados"
50         os.makedirs(self.save_folder, exist_ok=True)
51
52         # ----- Internal utilities -----
53         def _slice_attr(self, attr_arr, idx):
54             """Return attr_arr[idx] if it exists, otherwise None (e.g. feature_names_in_).
55             """
56             return attr_arr[idx] if attr_arr is not None else None
57
58         # ----- Main API -----
59         def create_lib(self, new_lib_name: str,
60                       selections: list[tuple[str, int]]):
61             """
62             Construct and save a new MinMaxScaler in the 'escalados' folder,
63             copying all internal attributes from the selected original scaler(s).
64
65             Available variables in each original scaler (1-based indices):
66
67             ICE_input:
68                 1 = Speed_rpm
69                 2 = m_fuel_mg
70                 3 = T_amb_K
71                 4 = p_amb_bar
72                 5 = Rack_Pos_1
73
74             ICE_output:
75                 1 = Torque_Nm
76                 2 = NO_out_1
77                 3 = CO_out_1
78
79             PG_input:
80                 1 = ICE_Speed_soll_rpm
81                 2 = EM2_Torque_Nm
82                 3 = ICE_Torque_Nm
83                 4 = Brake_perc
84
85             PG_output:
86                 1 = Car_Speed_kmph
87                 2 = SOC_1
88
89             Args:
90                 new_lib_name: Base name (without extension) for the .lib file.
91                 selections: List of (key, position_1based) tuples, where
92                             key is one of "ICE_input"/"ICE_output"/"PG_input"/"PG_output",
93                             and position_1based is the 1-based index of the variable.

```

```

93
94     Returns:
95         The assembled MinMaxScaler object with all internal attributes sliced.
96     """
97     # Pre-allocate arrays for new scaler attributes
98     cols      = len(selections)
99     scale      = np.empty(cols)
100     min_       = np.empty(cols)
101     data_min    = np.empty(cols)
102     data_max    = np.empty(cols)
103     data_range  = np.empty(cols)
104     feat_names = [] # may remain empty if names are not present
105
106     clip_val    = None
107     f_range     = None
108     n_samples   = None
109
110     # ---- Slice each selected column ----
111     for k, (key, pos1) in enumerate(selections):
112         src = self._av[key]
113         i   = pos1 - 1 # convert to 0-based index
114         scale[k] = src.scale_[i]
115         min_[k]  = src.min_[i]
116         data_min[k] = src.data_min_[i]
117         data_max[k] = src.data_max_[i]
118         data_range[k] = src.data_range_[i]
119
120         # Collect feature names if available
121         if hasattr(src, "feature_names_in_"):
122             feat_names.append(src.feature_names_in_[i])
123         else:
124             feat_names = None # drop names if any column lacks them
125
126         # Copy metadata once (shared across all columns)
127         clip_val = src.clip
128         f_range  = src.feature_range
129         n_samples = src.n_samples_seen_
130
131     # ---- Build the new scaler ----
132     new_scaler = MinMaxScaler(feature_range=f_range, clip=clip_val, copy=True)
133     # Manually assign sliced internal attributes
134     new_scaler.scale_ = scale
135     new_scaler.min_   = min_
136     new_scaler.data_min_ = data_min
137     new_scaler.data_max_ = data_max
138     new_scaler.data_range_ = data_range
139     new_scaler.n_features_in_ = cols
140     new_scaler.n_samples_seen_ = n_samples
141
142     if feat_names is not None:
143         new_scaler.feature_names_in_ = np.array(feat_names, dtype=object)
144
145     # ---- Save to disk ----
146     out_path = os.path.join(self.save_folder, f"{new_lib_name}.lib")
147     joblib.dump(new_scaler, out_path)
148     print(f"'{new_lib_name}.lib' saved in '{self.save_folder}/'")
149
150     return new_scaler

```

Listing 13: Code to combine multiple MinMaxScaler into a custom scaler collage

H Controller Training Script V2

```

1 import pandas as pd
2 import tensorflow as tf
3 import os
4 import random
5
6 def sample_init_state(folder_path="data"):
7     """

```

```

8 Returns a tf.Tensor of shape (6,) by sampling a random row
9 from a random CSV file in the specified folder and mapping
10 each column to its corresponding position.
11 """
12 csv_files = [f for f in os.listdir(folder_path) if f.endswith(".csv")]
13 if not csv_files:
14     raise FileNotFoundError(f"No CSV files found in {folder_path}")
15
16 chosen_file = os.path.join(folder_path, random.choice(csv_files))
17
18 # Count number of data rows (excluding header)
19 with open(chosen_file, "r") as f:
20     n_lines = sum(1 for _ in f) - 1
21 if n_lines <= 0:
22     raise ValueError(f"The file {chosen_file} is empty")
23
24 # Pick a random row index (0-based after header)
25 random_row_idx = random.randint(0, n_lines - 1)
26
27 # Read only that row
28 df_iter = pd.read_csv(
29     chosen_file,
30     skiprows=lambda x: x != 0 and x-1 != random_row_idx
31 )
32 row = df_iter.iloc[0]
33
34 # Extract columns and convert to tensors
35 mf = tf.constant(row["fuel"], dtype=tf.float32)
36 brk = tf.constant(row["Brake"], dtype=tf.float32)
37 ice_sp = tf.constant(row["ICE_Speed_soll"], dtype=tf.float32)
38 EM2 = tf.constant(row["EM2_Torque"], dtype=tf.float32)
39 torque = tf.constant(row["ICE_Torque_pred"], dtype=tf.float32)
40
41 state = tf.stack([mf, brk, ice_sp, EM2, torque], axis=0)
42 return state
43
44
45 def simulate_trajectory(controller,
46                         trans_func,
47                         init_state,
48                         clipping,
49                         K = 1200,
50                         vel_target = 70.0,
51                         warmup_steps = 200,
52                         alpha=1.0, beta=0.0, gamma=0.0):
53     """
54     Simulates a trajectory of K steps without training and
55     returns a DataFrame with all variables plus the average loss
56     from warmup_steps to K-1.
57     """
58     # Reset models and controller states
59     tf.py_function(trans_func.reset_models, [], [])
60     tf.py_function(controller.reset_states, [], [])
61
62     data = {k: [] for k in ("mf", "brk", "ice_sp", "torque",
63                            "vel_out", "nox", "co")}
64
65     # Constant ambient conditions
66     p_amb_bar = tf.constant(1.0, dtype=tf.float32)
67     T_amb_K = tf.constant(298.0, dtype=tf.float32)
68
69     # Initial state unpacking
70     mf, brk, ice_sp, _, torque = tf.unstack(init_state)
71     vel, _ = trans_func.predict_PG(ice_sp, 0.0, torque, brk)
72
73     # Loss accumulator
74     loss_total_last = tf.constant(0.0, dtype=tf.float32)
75
76     # Simulation loop
77     for i in range(K):
78         # Build controller input

```

```

79     x = tf.stack([vel_target, vel, mf, brk, ice_sp])
80     inp = tf.reshape(x, (1, 1, 5))
81
82     # Controller forward pass (no training)
83     out = controller(inp, training=False)[0]
84     mf, brk, ice_sp = tf.unstack(out, num=3)
85
86     # ICE prediction (returns torque, nox, _, co, _)
87     torque, nox, _, co, _ = trans_func.predict_ice(
88         ice_sp, mf, T_amb_K, p_amb_bar
89     )
90     if clipping:
91         torque = tf.clip_by_value(torque, -50.0, 300.0)
92
93     # Zero torque below idle threshold
94     torque = tf.where(ice_sp < 900.0, 0.0, torque)
95     vel_out, _ = trans_func.predict_PG(ice_sp, 0.0, torque, brk)
96
97     # Accumulate loss only after warm-up
98     if i >= warmup_steps:
99         step_loss = (
100             alpha * tf.square(vel_target - vel_out)
101             + beta * nox + gamma * co
102         )
103         loss_total_last += step_loss
104
105     # Record data
106     data["mf"].append(mf.numpy())
107     data["brk"].append(brk.numpy())
108     data["ice_sp"].append(ice_sp.numpy())
109     data["torque"].append(torque.numpy())
110     data["vel_out"].append(vel_out.numpy())
111     data["nox"].append(nox.numpy())
112     data["co"].append(co.numpy())
113
114     vel = vel_out # next step
115
116     # Compute average loss over post warmup steps
117     effective_steps = K - warmup_steps
118     total_loss = loss_total_last / tf.cast(effective_steps, tf.float32)
119     tf.print("total_loss =", total_loss)
120
121     df = pd.DataFrame(data)
122     df["step"] = df.index
123     return df
124
125
126 def create_interactive_plot(df, title):
127     fig = px.line(
128         df,
129         x="step",
130         y=["mf", "brk", "ice_sp", "torque", "vel_out", "nox", "co"],
131         title=title
132     )
133     # Hide output traces by default
134     for tr in fig.data:
135         if tr.name in ("vel_out", "nox", "co"):
136             tr.visible = "legendonly"
137
138     buttons = [
139         dict(
140             label="States",
141             method="update",
142             args=[
143                 {"visible": [t.name in ("mf", "brk", "ice_sp", "torque") for t in fig.
144 data]},
145                 {"title": f"{title} States"}
146             ],
147         ),
148         dict(
149             label="Outputs",

```

```

149         method="update",
150         args=[
151             {"visible": [t.name in ("vel_out", "nox", "co") for t in fig.data]},
152             {"title": f"{title} Outputs"}
153         ]
154     ),
155 ]
156 fig.update_layout(
157     updatemenus=[dict(type="buttons", buttons=buttons, direction="right")]
158 )
159 return fig
160
161
162 #####
163 # eval_loss_components_fast optimized version ( 1015 faster on GPU)
164 #####
165 @tf.function()
166 def eval_loss_components_fast(controller,
167                               trans_func,
168                               N = 20,
169                               total_steps = 1200,
170                               last_steps = 200, # ignored but kept
171                               clipping = True,
172                               vel_target = 75.0,
173                               warmup_steps = 200):
174     """
175     Evaluates N trajectories WITHOUT training and returns the averages of:
176     1) |vel_target vel_out|
177     2) NOx
178     3) CO
179
180     Accumulation starts only after `warmup_steps`.
181     Averages are divided by (total_steps warmup_steps).
182     """
183     # Constants
184     p_amb_bar = tf.constant(1.0, tf.float32)
185     T_amb_K = tf.constant(298.0, tf.float32)
186
187     # Global accumulators
188     g_vel, g_nox, g_co = 0.0, 0.0, 0.0
189
190     for _ in tf.range(N):
191         # Reset models and states
192         tf.py_function(trans_func.reset_models, [], [])
193         tf.py_function(controller.reset_states, [], [])
194
195         # Random initial state
196         init_state = sample_init_state()
197         mf, brk, ice_sp, EM2, torque = tf.unstack(init_state)
198         vel, _ = trans_func.predict_PG(ice_sp, EM2, torque, brk)
199
200         # Per-trajectory accumulators
201         t_vel, t_nox, t_co = 0.0, 0.0, 0.0
202
203         for i in tf.range(total_steps):
204             x = tf.stack([vel_target, vel, mf, brk, ice_sp])
205             inp = tf.reshape(x, (1, 1, 5))
206
207             out = controller(inp, training=False)[0]
208             mf, brk, ice_sp = tf.unstack(out, num=3)
209
210             torque, nox, _, co, _ = trans_func.predict_ice(
211                 ice_sp, mf, T_amb_K, p_amb_bar
212             )
213             if clipping:
214                 torque = tf.clip_by_value(torque, -50.0, 300.0)
215
216             torque = tf.where(ice_sp < 900.0, 0.0, torque)
217             vel_out, _ = trans_func.predict_PG(ice_sp, 0.0, torque, brk)
218
219             if i >= warmup_steps:

```

```

220         t_vel += tf.abs(vel_target - vel_out)
221         t_nox += nox
222         t_co += co
223
224         vel = vel_out
225
226         inv_steps = 1.0 / tf.cast(total_steps - warmup_steps, tf.float32)
227         g_vel += t_vel * inv_steps
228         g_nox += t_nox * inv_steps
229         g_co += t_co * inv_steps
230
231         tf.print("trajectory loss =", t_vel * inv_steps)
232
233         inv_N = 1.0 / tf.cast(N, tf.float32)
234         return g_vel * inv_N, g_nox * inv_N, g_co * inv_N
235
236
237 import tensorflow as tf
238 from tensorflow.keras import layers
239 import tensorflow_addons as tfa
240
241 class ScaledController(tf.keras.Model):
242     def __init__(self,
243                 scaler_params: dict,
244                 units: int = 128,
245                 alpha: float = 100.0):
246
247         super().__init__(name="scaled_controller")
248
249         # Extract scaler parameters
250         dmin = scaler_params['data_min'].astype('float32')
251         dmax = scaler_params['data_max'].astype('float32')
252         scale = scaler_params['scale'].astype('float32')
253         min_tf = scaler_params['min'].astype('float32')
254
255         # TensorFlow constants
256         self._min_tf = tf.constant(min_tf)
257         self._scale = tf.constant(scale)
258         self._max = tf.constant(dmax)
259         self._range = self._max - tf.constant(dmin)
260
261         # Physical limits in normalized space
262         phys_lower = tf.constant([3.0, 0.0, 0.0], tf.float32)
263         phys_upper = tf.constant([70.0, 100.0, 4500.0], tf.float32)
264
265         # Compute normalized bounds
266         self.lower_norm = phys_lower * self._scale[2:] + self._min_tf[2:]
267         self.upper_norm = phys_upper * self._scale[2:] + self._min_tf[2:]
268
269         # Normalized idle threshold
270         self.tau_norm = 900.0 * self._scale[4] + self._min_tf[4]
271         self.alpha = alpha
272
273         # LayerNorm LSTM cell
274         ln_cell = tfa.rnn.LayerNormLSTMCell(units)
275         self.lstm = tf.keras.layers.RNN(
276             ln_cell, stateful=True, return_sequences=False
277         )
278
279         self.dense = layers.Dense(units // 2, activation="tanh")
280         self.delta = layers.Dense(3, activation="linear", name="deltas")
281
282     def _to_norm(self, x):
283         return x * self._scale + self._min_tf
284
285     def _from_norm(self, n):
286         return (n - self._min_tf) / self._scale
287
288     def call(self, inputs):
289         x = tf.squeeze(inputs, axis=1) # (batch,5)
290         x_norm = self._to_norm(x) # (batch,5)

```



```

291         h = self.lstm(tf.expand_dims(x_norm, 1))
292         h = self.dense(h)
293         deltas = self.delta(h) # (batch,3)
294
295         orig = x_norm[:, 2:] # (batch,2)
296         abs_raw = orig + deltas
297
298         # Soft bounding via sigmoid
299         bounded = (
300             self.lower_norm +
301             (self.upper_norm - self.lower_norm) * tf.sigmoid(abs_raw)
302         )
303
304         # Gate for ice_sp < idle threshold
305         ice_norm = bounded[:, 2:3]
306         gate = tf.sigmoid((ice_norm - self.tau_norm) * self.alpha)
307
308         # Denormalize everything
309         full_norm = tf.concat([x_norm[:, :2], bounded], axis=1)
310         full_phys = self._from_norm(full_norm) # in physical space
311
312         mf_phys = full_phys[:, 2:3] * gate # zero if gate=0
313         rest_phys = full_phys[:, 3:] # brk_phys, ice_sp_phys
314
315         # Return 3 physical outputs: mf, brk, ice_sp
316         return tf.concat([mf_phys, rest_phys], axis=1)
317
318     def reset_states(self):
319         self.lstm.reset_states()
320
321
322 @tf.function
323 def rollout_and_loss(controller, trans_func, init_state,
324                     alpha, beta, gamma, opt, clipping,
325                     total_steps=1200, last_steps=200,
326                     warmup_steps=10, vel_target=70.0,
327                     training=True):
328     """
329     Simulates 1200 steps with on-the-fly training and returns
330     the average loss over only the last 200 steps.
331     """
332     # Reset ICE/PG and controller states
333     tf.py_function(trans_func.reset_models, [], [])
334     tf.py_function(controller.reset_states, [], [])
335
336     # Constant ambient conditions
337     p_amb_bar = tf.constant(1.0, tf.float32)
338     T_amb_K = tf.constant(298.0, tf.float32)
339
340     # Unpack initial state
341     mf, brk, ice_sp, EM2, torque = tf.unstack(init_state)
342     vel, _ = trans_func.predict_PG(ice_sp, EM2, torque, brk)
343
344     loss_total_last = tf.constant(0.0, tf.float32)
345
346     for i in tf.range(total_steps):
347         # Build controller input
348         x = tf.stack([vel_target, vel, mf, brk, ice_sp])
349         inp = tf.reshape(x, (1,1,5))
350
351         with tf.GradientTape() as tape:
352             out = controller(inp, training=training)[0]
353             mf, brk, ice_sp = tf.unstack(out, num=3)
354
355             # Differentiable ICE + PG prediction
356             torque, nox, _, co, _ = trans_func.predict_ice(
357                 ice_sp, mf, T_amb_K, p_amb_bar
358             )
359             if clipping:
360                 torque = tf.clip_by_value(torque, -50.0, 300.0)

```

```

362     mask_ice = ice_sp < 900.0
363     torque   = tf.where(mask_ice, tf.zeros_like(torque), torque)
364
365     vel_out, _ = trans_func.predict_PG(ice_sp, 0.0, torque, brk)
366
367     # Compute step loss
368     step_loss = (
369         alpha * tf.square(vel_target - vel_out)
370         + beta * nox + gamma * co
371     )
372     # Warm-up mask
373     w = tf.minimum(1.0, tf.cast(i, tf.float32) / tf.cast(warmup_steps, tf.
374 float32))
375     step_loss *= w
376
377     # Apply gradients immediately
378     grads = tape.gradient(step_loss, controller.trainable_weights)
379     opt.apply_gradients(zip(grads, controller.trainable_weights))
380
381     # Accumulate loss after warm-up
382     if i >= warmup_steps:
383         loss_total_last += step_loss
384
385     vel = vel_out
386
387     avg_loss = loss_total_last / tf.cast(total_steps - warmup_steps, tf.float32)
388     return avg_loss
389
390 def train_and_save_controller(epochs,
391                               alpha, beta, gamma,
392                               name,
393                               ruta_ICE, ruta_PG,
394                               controller,
395                               learning_rate=1e-4,
396                               output_root="models",
397                               clipping=False,
398                               evaluate=False,
399                               decay=False,
400                               plateau_patience=20,
401                               plateau_factor=0.5,
402                               early_stop_patience=50,
403                               restore_on_plateau=True,
404                               vel_target=70.0,
405                               warmup_steps=200,
406                               good_loss=10.0):
407
408     """
409     Trains the controller with:
410     - Manual ReduceLROnPlateau
411     - Early stopping
412     - (Optional) Restoring best weights before reducing LR
413
414     Returns the output folder and list of epoch losses.
415     """
416     # Create output folder
417     folder = os.path.join(
418         output_root,
419         f"{name}_{vel_target}velTarget_{warmup_steps}warmup_"
420         f"{good_loss}good_loss_{epochs}ep_{alpha:.2f}a_{beta:.2f}b_{gamma:.2f}g"
421     )
422     os.makedirs(folder, exist_ok=True)
423
424     # TensorFlow constants
425     a = tf.constant(alpha, dtype=tf.float32)
426     b = tf.constant(beta, dtype=tf.float32)
427     g = tf.constant(gamma, dtype=tf.float32)
428
429     # Initialize transition model and optimizer
430     trans_func = setup_transition_function_model(ruta_ICE, ruta_PG)
431     opt         = tf.keras.optimizers.Adam(learning_rate, clipnorm=0.5)

```

```

432 last_steps = 1200 - warmup_steps
433
434
435 # Short multi-start for weight initialization
436 seeds = [0,1,2]
437 short_epochs = 5
438 best_init_loss = float('inf')
439 best_init_weights = None
440
441 for seed in seeds:
442     tf.random.set_seed(seed)
443     losses_tmp = []
444     for _ in range(short_epochs):
445         st = sample_init_state()
446         Ls = rollout_and_loss(
447             controller, trans_func, st,
448             a, b, g, opt, clipping,
449             vel_target=vel_target,
450             warmup_steps=warmup_steps,
451             last_steps=last_steps,
452             training=False
453         )
454         losses_tmp.append(Ls.numpy())
455     avg = sum(losses_tmp) / short_epochs
456     print(f"[Seed {seed}] short loss: {avg:.4f}")
457     if avg < best_init_loss:
458         best_init_loss = avg
459         best_init_weights = controller.get_weights()
460
461 controller.set_weights(best_init_weights)
462 print(f"    Using seed with best short loss: {best_init_loss:.4f}")
463
464 # Training loop with plateau and early stopping
465 best_loss = float('inf')
466 best_weights = None
467 wait = 0
468 es_wait = 0
469
470 losses = []
471 for ep in range(epochs):
472     init_state = sample_init_state()
473     L = rollout_and_loss(
474         controller, trans_func, init_state,
475         a, b, g, opt, clipping,
476         vel_target=vel_target,
477         warmup_steps=warmup_steps,
478         last_steps=last_steps
479     )
480     losses.append(L.numpy())
481     print(f"Epoch {ep+1}/{epochs}    Avg loss: {L:.4f}")
482
483     if L < best_loss:
484         best_loss = L
485         best_weights = controller.get_weights()
486         wait = es_wait = 0
487     else:
488         wait += 1
489         es_wait += 1
490
491     # Early stopping
492     if es_wait >= early_stop_patience:
493         print(f"    Early stopping: no improvement in {early_stop_patience}
494 epochs.")
495         break
496
497     # Reduce LR on plateau
498     if wait >= plateau_patience:
499         if restore_on_plateau and best_weights is not None:
500             controller.set_weights(best_weights)
501             print("    Restored best weights before reducing LR.")
502         old_lr = opt.learning_rate.numpy()

```

```

502         new_lr = old_lr * plateau_factor
503         opt.learning_rate.assign(new_lr)
504         print(f"          No improvement in {plateau_patience} epochs      LR {
old_lr:.2e}          {new_lr:.2e}")
505         wait = 0
506
507     # Dynamic warm-up adjustment
508     if L.numpy() <= good_loss and warmup_steps > 5:
509         new_ws = max(5, warmup_steps - 10)
510         if new_ws != warmup_steps:
511             print(f"          warmup_steps {warmup_steps}          {new_ws} (reset LR)")
512             warmup_steps = new_ws
513             last_steps = 1200 - warmup_steps
514             opt.learning_rate.assign(learning_rate)
515             wait = es_wait = 0
516             best_loss = float('inf')
517
518     # Restore best weights before saving
519     if best_weights is not None:
520         controller.set_weights(best_weights)
521
522     # Save final model
523     model_path = os.path.join(folder, name)
524     controller.save(model_path, include_optimizer=False, save_format="tf")
525     print(f"          Model saved to {model_path}")
526
527     # Plot training loss
528     plt.figure()
529     plt.plot(range(1, len(losses)+1), losses)
530     plt.xlabel("Epoch")
531     plt.ylabel("Average Loss")
532     plt.title(f"Training Loss ({name})")
533     plt.grid(True)
534     plt.tight_layout()
535     plot_path = os.path.join(folder, "loss.png")
536     plt.savefig(plot_path)
537     plt.close()
538     print(f"          Loss plot saved to {plot_path}")
539
540     # Final trajectory plot
541     df = simulate_trajectory(controller, trans_func, init_state, clipping, vel_target
=vel_target)
542     fig = create_interactive_plot(df, "Final Trajectory")
543     html_path = os.path.join(folder, "traj_final.html")
544     fig.write_html(html_path)
545
546     if evaluate:
547         mean_vel, mean_nox, mean_co = eval_loss_components_fast(
548             controller, trans_func,
549             N=10, total_steps=1200,
550             last_steps=last_steps,
551             clipping=clipping,
552             vel_target=vel_target,
553             warmup_steps=warmup_steps
554         )
555         print(f"          Average loss over last {1200-warmup_steps} steps      ")
556         print(f"|vel error|: {mean_vel.numpy():.4f}")
557         print(f"NOx          : {mean_nox.numpy():.4f}")
558         print(f"CO           : {mean_co.numpy():.4f}")
559
560     print(f"          Interactive plot saved to {html_path}")
561
562     return folder, losses

```

Listing 14: Controller Training v2